

コンピュータ アーキテクチャ I 第9回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

課題の解説

考え方の指針

1. まず理想的にプログラムが実行された場合のサイクル数を考える
2. 何かイヤなイベント（ハザード）が起きる回数を計算する
 - 全体の命令数，命令の種類毎の出現率，イベントの発生確率などをかけていけば良い
3. イベントによる総延長時間を求める
 - イベントの発生回数と，イベント 1 回あたり何サイクル実行時間が伸びるかをかけて求める
4. 1. の理想的な実行サイクル数に総延長時間を加えて実際の実行サイクル数を求める
5. 実行命令数を 4. で求めたサイクル数で割る

分岐予測ミスによる実行サイクルの増加のモデル

- 以下のようにおいた場合,
 - ミスがない理想的な実行の際のサイクル数 : C_t
 - 分岐予測ミスの発生回数 : $N_m = N_i \times P_b \times P_m$
 - 実行命令数 : N_i
 - プログラム中の分岐命令の出現率 : P_b
 - 分岐命令毎の予測ミス発生率 : P_m
 - 分岐予測ミス・ペナルティ : C_p
- 実行サイクル数 C_r は, 理想サイクル数 C_t に対して,
 - $C_r = C_t + N_m \times C_p$
 - 意味 : 予測ミスの発生回数×ペナルティ だけ実行時間が伸びる

具体的な値を入れてみる

■ 以下のように置いた場合,

- 理想的な実行の際のサイクル数 : $Ct = 1M$ (メガ)
- 分岐予測ミスの発生回数 : $Nm = Ni \times Pb \times Pm = 0.025M$
 - 実行命令数 : $Ni = 1M$
 - 分岐命令の出現率 : $Pb = 0.25$
(プログラムは大体 4 命令に 1 つぐらい分岐が出てくる)
 - 分岐命令毎の予測ミス発生率 : $Pm = 0.1$
- 分岐予測ミス・ペナルティ : $Cp = 4$

■ 実行サイクル数 Cr は

- $Cr = Ct + Nm \times Cp = 1M + 0.1M = 1.1M$
- 分岐予測ミスにより 10% 実行サイクル数が伸びている

IPC で考えると

- 最終的な性能を考える上で IPC の方が都合がよい

- 実行サイクル数 C_r を命令数 N_i で正規化すると,

- $$\frac{C_r}{N_i} = \frac{C_t}{N_i} + \frac{(N_m \times C_p)}{N_i} = \frac{C_t}{N_i} + \frac{(N_i \times P_b \times P_m \times C_p)}{N_i} = \frac{C_t}{N_i} + P_b \times P_m \times C_p$$

- IPC は命令数を実行サイクル数で割ったもの = つまり上記の逆数

- $$IPC_r = \frac{1}{\frac{C_t}{N_i} + P_b \times P_m \times C_p} = \frac{1}{\frac{1}{IPC_t} + P_b \times P_m \times C_p}$$

- ここで IPC_r は実際の IPC, IPC_t は理想 IPC

一般化できる

- 以下のようにおいた場合,
 - 理想的な実行の際のサイクル数 : C_t
 - 何らかのハザードの発生回数 : $N_h = N_i \times P_i \times P_h$
 - 実行命令数 : N_i
 - ハザードを起こす命令の出現率 : P_i
 - その命令毎のハザード発生率 : P_h
 - ハザード時のサイクル数の増加 : C_p
- 実行サイクル数 C_r は, 理想サイクル数 C_t に対して,
 - $C_r = C_t + N_h \times C_p$

一般化できる

- ハザード a, ハザード b, ハザード c ... とあった時
 - それぞれおきた回数が $Nh..$, ペナルティが $Cp ...$ とする
- 実行サイクル数 Cr は, 理想サイクル数 Ct に対して,
 - $Cr = Ct + Nha \times Cpa + Nhb \times Cpb + Nhc \times Cpc + \dots$
- 実行命令数を Ni とすると, $IPC_r = Ni/Cr$

課題 8

- 以下のような条件を考える
 - 10段のパイプラインを持つ 2-way スーパースカラプロセッサであり, 理想的には $IPC=2$ で実行できる
 - 全実行命令におけるなんらかのデータハザードの発生率は 0.2
 - このデータハザード発生時は 1 サイクル実行時間が伸びるものとする
 - 全実行命令における分岐命令の出現率は 0.2
 - 分岐予測ミス率は 0.3
- (1) この CPU を改良する際,
 - 「3-way スーパースカラにする」
 - 「2-way のまま15段パイプラインにする」
 - 「2-way のまま分岐予測器を改良する」のどれが最も性能が上がるかを性能を計算して検討せよ
 - この CPU を 3-way スーパースカラにすると理想的には $IPC=3$ で実行できるがデータハザードの発生率は 0.3 に上昇するとする
 - また, 分岐予測器を改良すると分岐予測ミス率が 0.2 にまで削減されるとする

課題8 (1) まず実行サイクル数を求めてから, 正規化する

- 以下のような条件を考える

- 10段のパイプラインを持つ 2-way スーパースcalarプロセッサであり, 理想的には IPC=2 で実行できる
- 全実行命令におけるなんらかのデータハザードの発生率は 0.2
- このデータハザード発生時は 1 サイクル実行時間が伸びるものとする
- 全実行命令における分岐命令の出現率は 0.2
- 分岐予測ミス率は 0.3

- 上記のベースライン CPU の 実行サイクル数 C_r は, 実行命令数を N_i とすると

- $C_r = C_t + N_{ma} \times C_{pa} + N_{mb} \times C_{pb}$
- $= \frac{N_i}{IPC_t} + (N_i \times 0.2) \times 1 + (N_i \times 0.2 \times 0.3) \times 9$
- $= \frac{N_i}{2} + N_i \times 0.2 + N_i \times 0.54 = 1.24 \times N_i$

- IPC_r は N_i を C_r で正規化したものなので,

- $IPC_r = \frac{N_i}{C_r} = \frac{N_i}{1.24 \times N_i} \approx 0.81$

課題 8 (1)

- ベースライン CPU の動作周波数を 1 とすると, その性能は
周波数 $1 \times \text{IPC } 0.81 = 0.81$
- 「3-way スーパスカラにする」
 - $$\text{IPC}r = \frac{1}{\frac{1}{3} + 0.3 \times 1 + 0.2 \times 0.3 \times 9} \approx \frac{1}{1.17} \approx 0.85$$
 - 先ほどは丁寧にサイクル数を 1 回計算したが,
ここでは最初から「1 命令あたりのサイクル数」
として考えて IPC をいきなり求めている (結果は同じ)
 - 周波数は変わらないなら性能も 0.85

課題 8 (1)

■ 「2-way のまま15段パイプラインにする」

- $IPC_r = \frac{1}{\frac{1}{2} + 0.2 \times 1 + 0.2 \times 0.3 \times 14} \approx \frac{1}{1.54} \approx 0.65$

- 分岐予測ミスペナルティが 10 から 15 に

- 10 → 15 段になったことにより周波数が 1.5 倍に

- 性能は $1.5 \times 0.65 = 0.97$

■ 「2-way のまま分岐予測器を改良する」

- $IPC_r = \frac{1}{\frac{1}{2} + 0.2 \times 1 + 0.2 \times 0.2 \times 9} = \frac{1}{1.1} \approx 0.94$

- 周波数は変わらないので, 性能も 0.94

課題 8 (1)

- まとめると,
 - ベースライン : 0.81
 1. 3-way スーパスカラにする : 0.85
 2. 2-way のまま15段パイプラインにする : 0.97 最も速い
 3. 2-way のまま分岐予測器を改良する : 0.94

課題 8 (2)

- (2) ここで効率を表す「性能エネルギー比 (性能 / 消費エネルギー)」という指標を導入する. この数字が大きいほど小さなエネルギーで速く動くことを意味する. 前述の 3 つの改良方針のうち, 以下の追加の条件のもとで, 最も性能エネルギー比が良いものはどれかを計算して検討せよ.
 - クロック周波数を N 倍にすると, 消費エネルギーは N^2 倍増える
 - 3-way スーパースカラにすると消費エネルギーは 1.5 倍になる
 - 分岐予測器を改良すると消費エネルギーは 1.1 倍になる

課題 8 (2)

■ ベースラインの消費エネルギーを 1 とすると : $0.81/1=0.81$

1. 3-way スーパスカラにする : $0.85/1.5 \approx 0.57$

2. 2-way のまま15段パイプラインにする : $\frac{0.97}{1.5^2} \approx 0.43$

- 最も速いが, エネルギー効率がとても悪い

3. 2-way のまま分岐予測器を改良する : $\frac{0.94}{1.1} \approx 0.85$

- 2 より少し遅いがエネルギー効率はずっと良い

メモリと、キャッシュの基本

今日の内容

1. メモリ
2. キャッシュの基本

メモリ

メモリ

- メモリ：RAM（Random Access Memory）
 - 複数のデータを記憶する回路
 - 配列のように、位置を指定して読み書きする

メモリ

データ：	07h	10h	30h	...	3Eh	01h	32h	20h	80h	...
アドレス：	0h	1h	2h	...	8000h	8001h	8002h	8003h	8004h	...

- メモリは命令列と、計算するデータを保持する
 - 単一の巨大な配列があると思えばよい
 - C 言語の配列は、これを切り出してユーザーに見せている
- 数字が入る箱がたくさん並んでいるイメージ
 - アドレス：箱の通し番号（住所）
 - データ：箱の中身の数字

メモリ

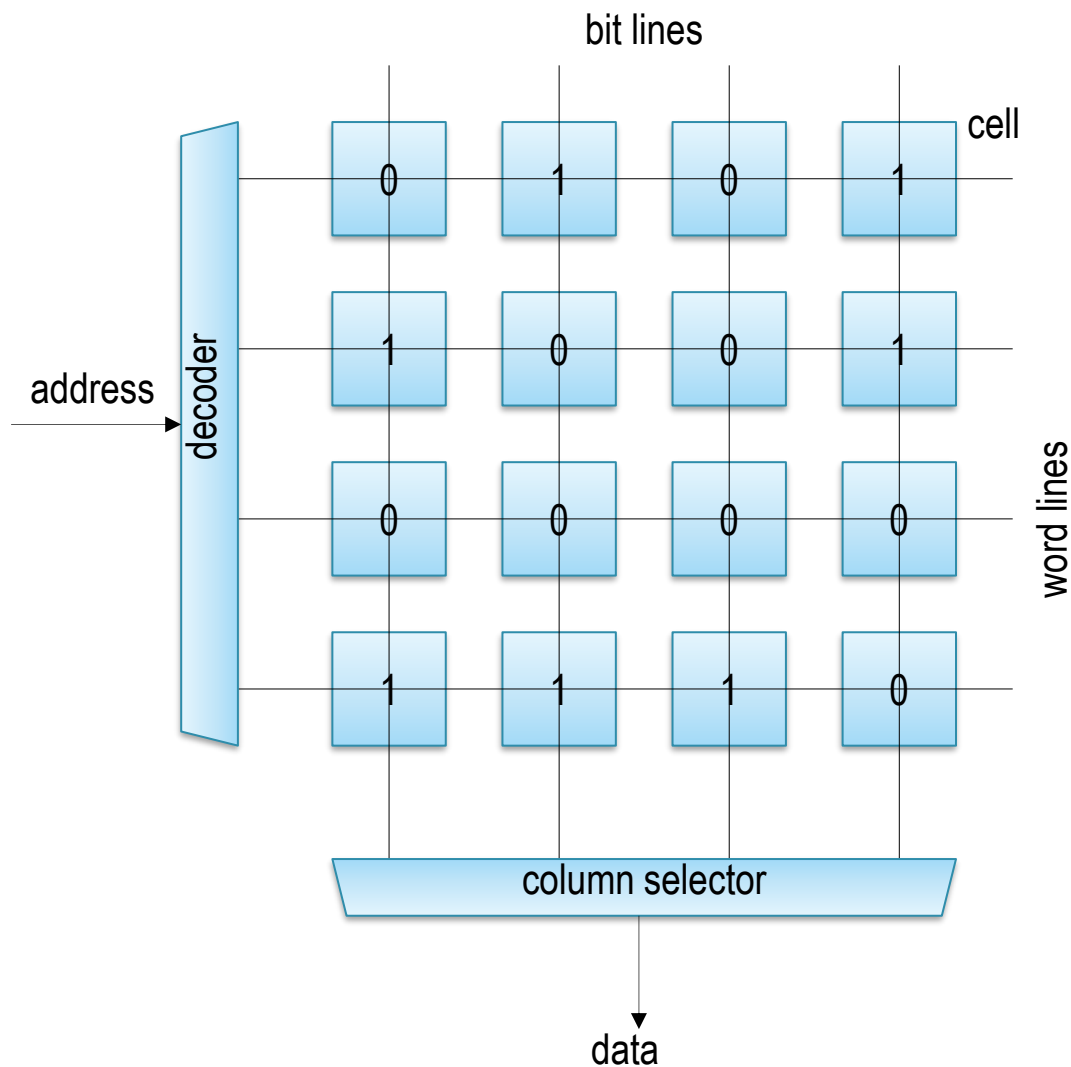
1. メモリの基本

1. 構造
2. 動作
3. 容量と速度

2. メモリの詳細

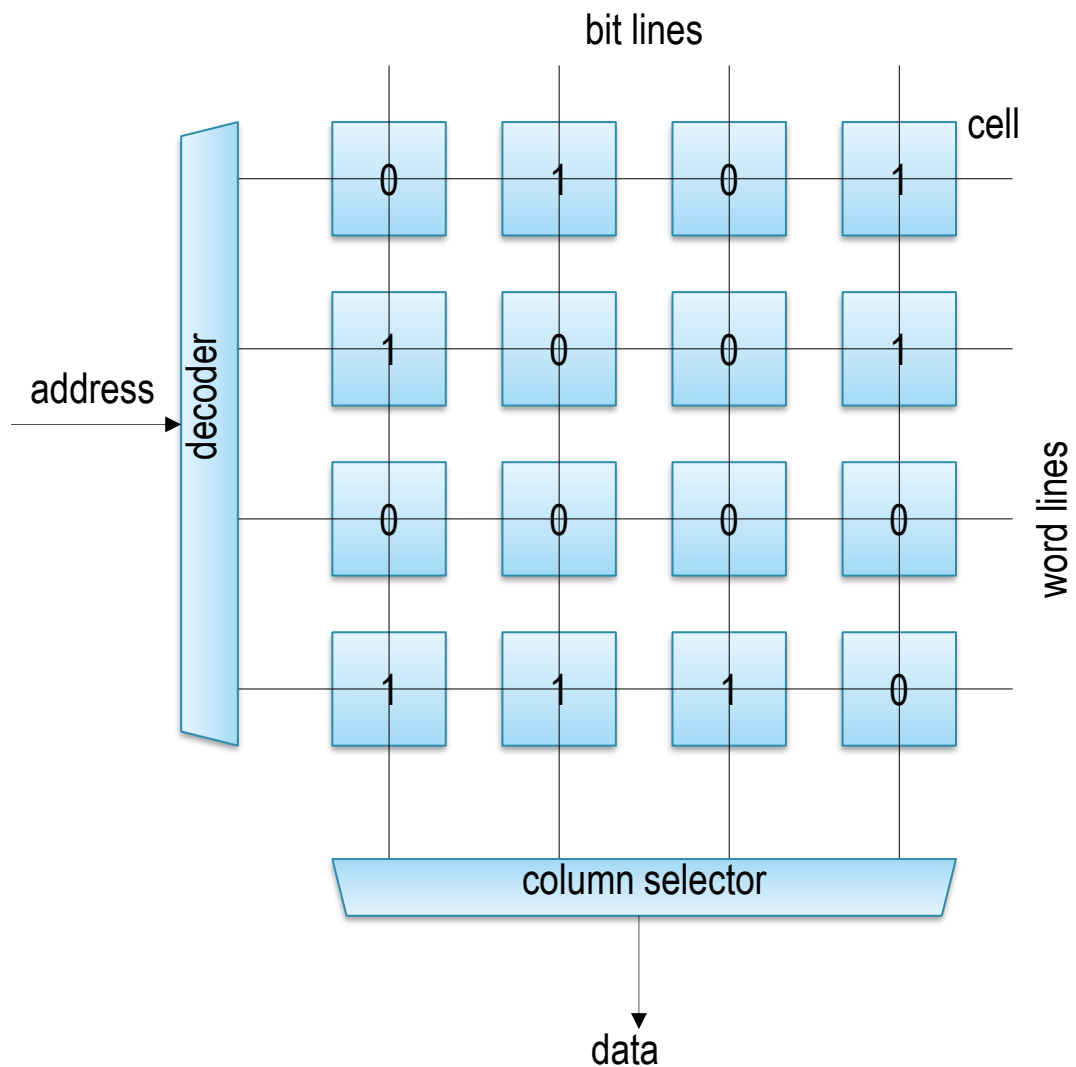
1. SRAM と DRAM
2. なぜメモリが存在するのか？

メモリの基本構造：セルを行列状に配置



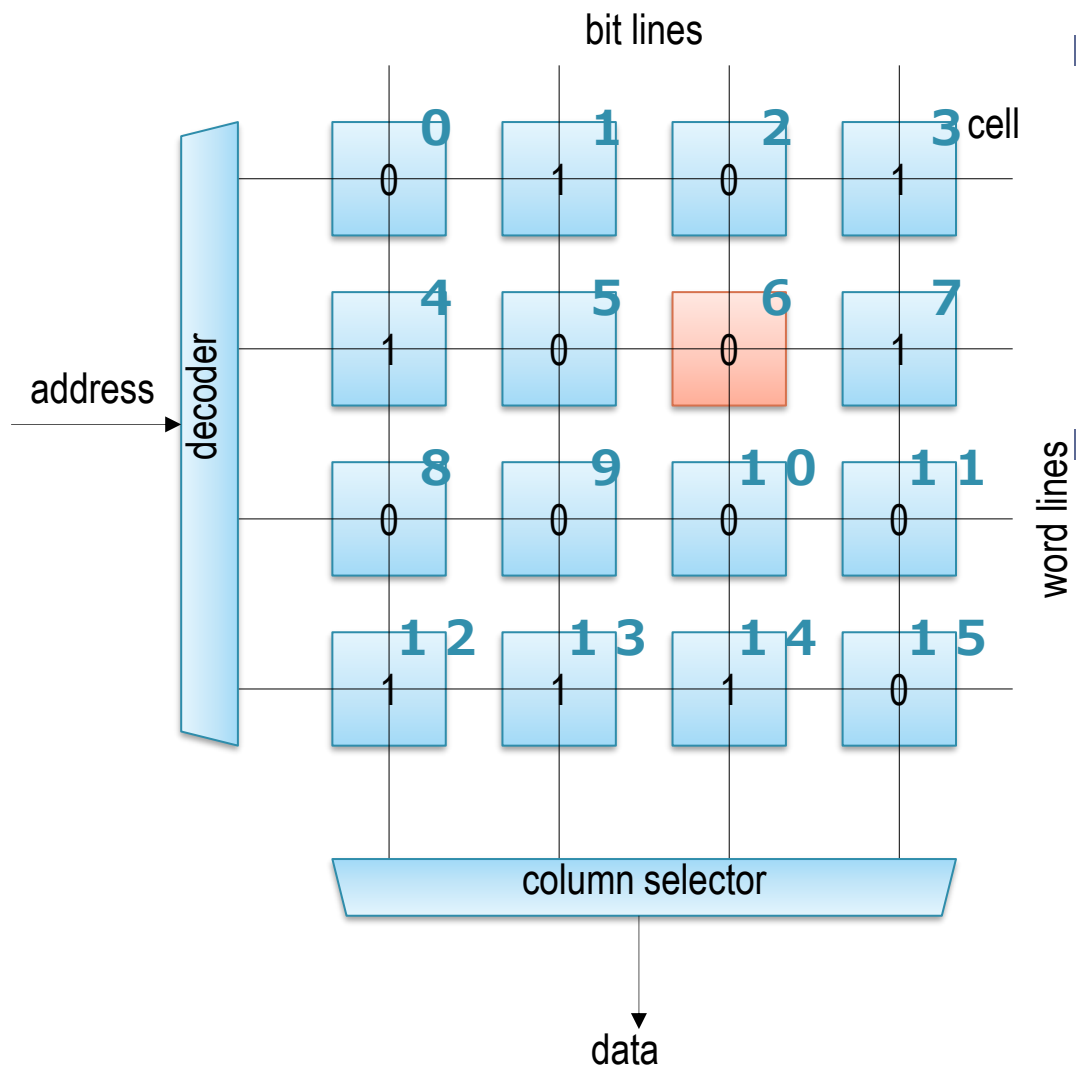
- セル：
 - 1ビットの情報を記憶
- ビットライン：
 - 列方向の配線
- ワードライン：
 - 行方向の配線
- デコーダ：
 - アドレスをデコードしてワードラインをアサート
- カラム・セクタ：
 - ビットライン出力から1つを選んで出力

メモリの読み出し操作



1. アドレスのデコード
 2. ワードラインのアサート (行の指定)
 3. ビットラインへの1行分のデータの読み出し
 4. カラムセクタによる目的のビットの選択
- (書き込みはこれの逆を行う)

メモリの読み出し動作の例



■ 要素数 $16 = 2^4$

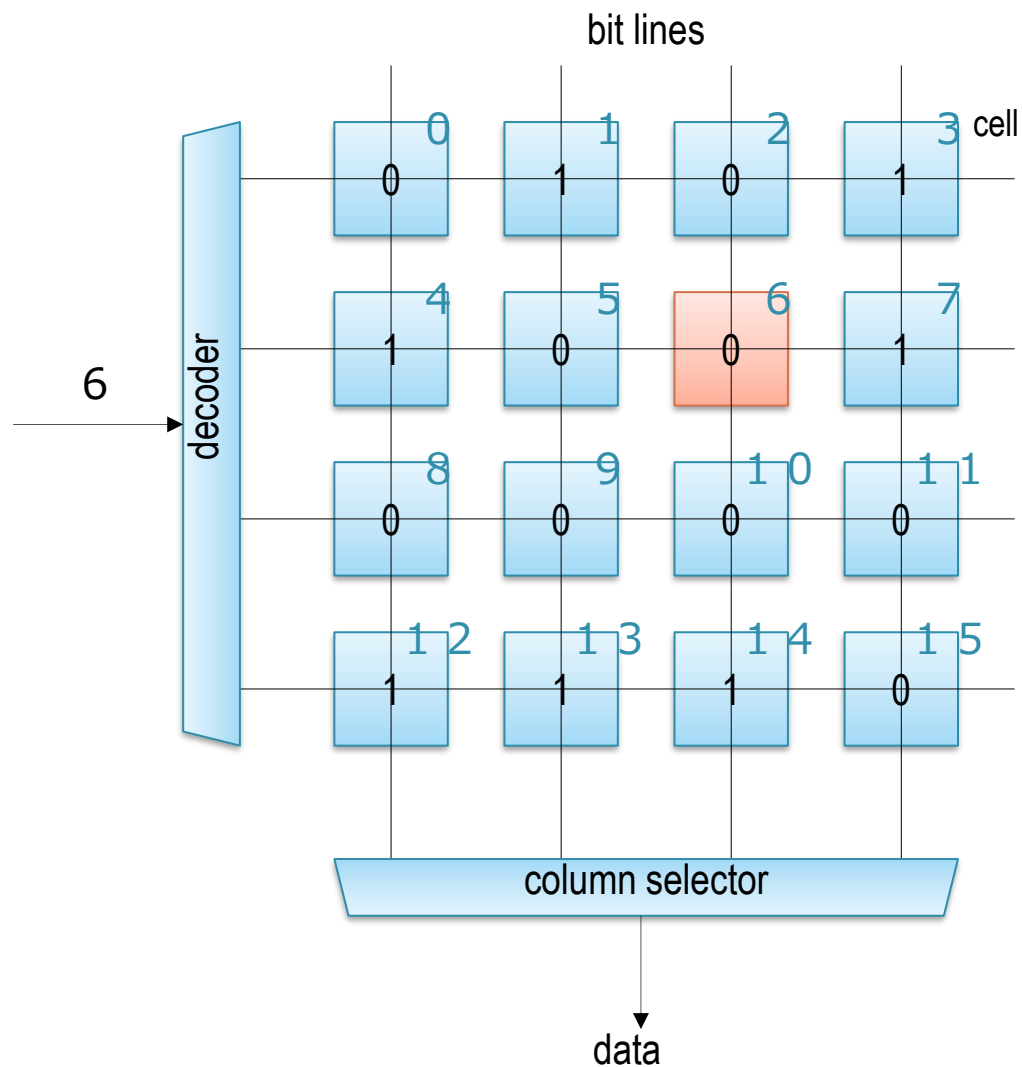
- アドレスは4ビット
- 各セルの右上の数字がアドレス

■ このメモリの読み出し操作を例として説明

- アドレス6のセルを読み出す

メモリの読み出し動作 (1)

アドレスのデコード



■ どの行を読むか決める

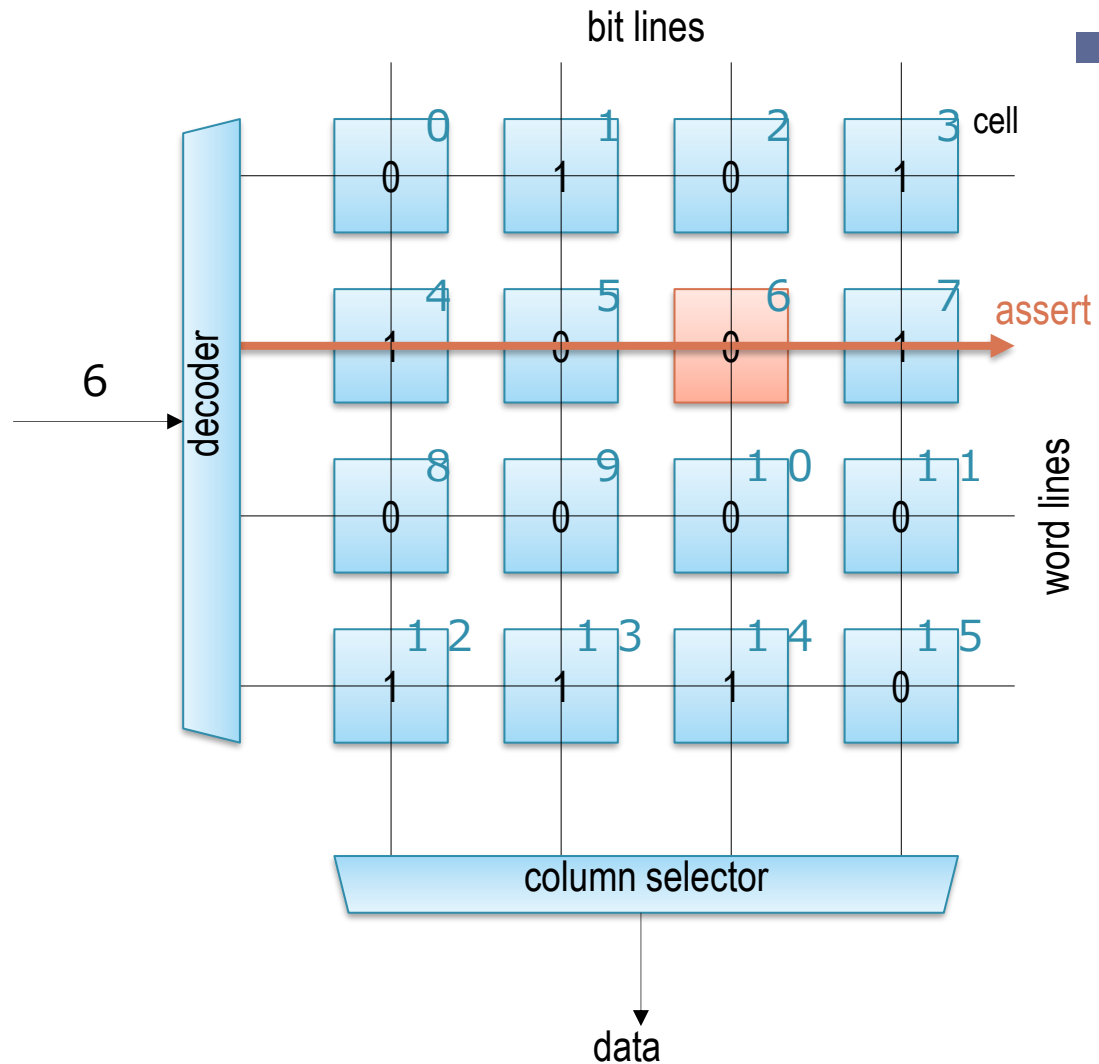
- 各行は4つセルがある
- アドレスを4で割って切り捨てた行目が読むべき場所
- $6 = 0110$ (2進数)

■ デコーダ

- 数字を対応するワンホット信号に変換する回路
- ワンホット信号：
 - n 本のうち、1つだけが1で他が0の信号
- アドレス上位の2ビットをデコード

メモリの読み出し動作 (2)

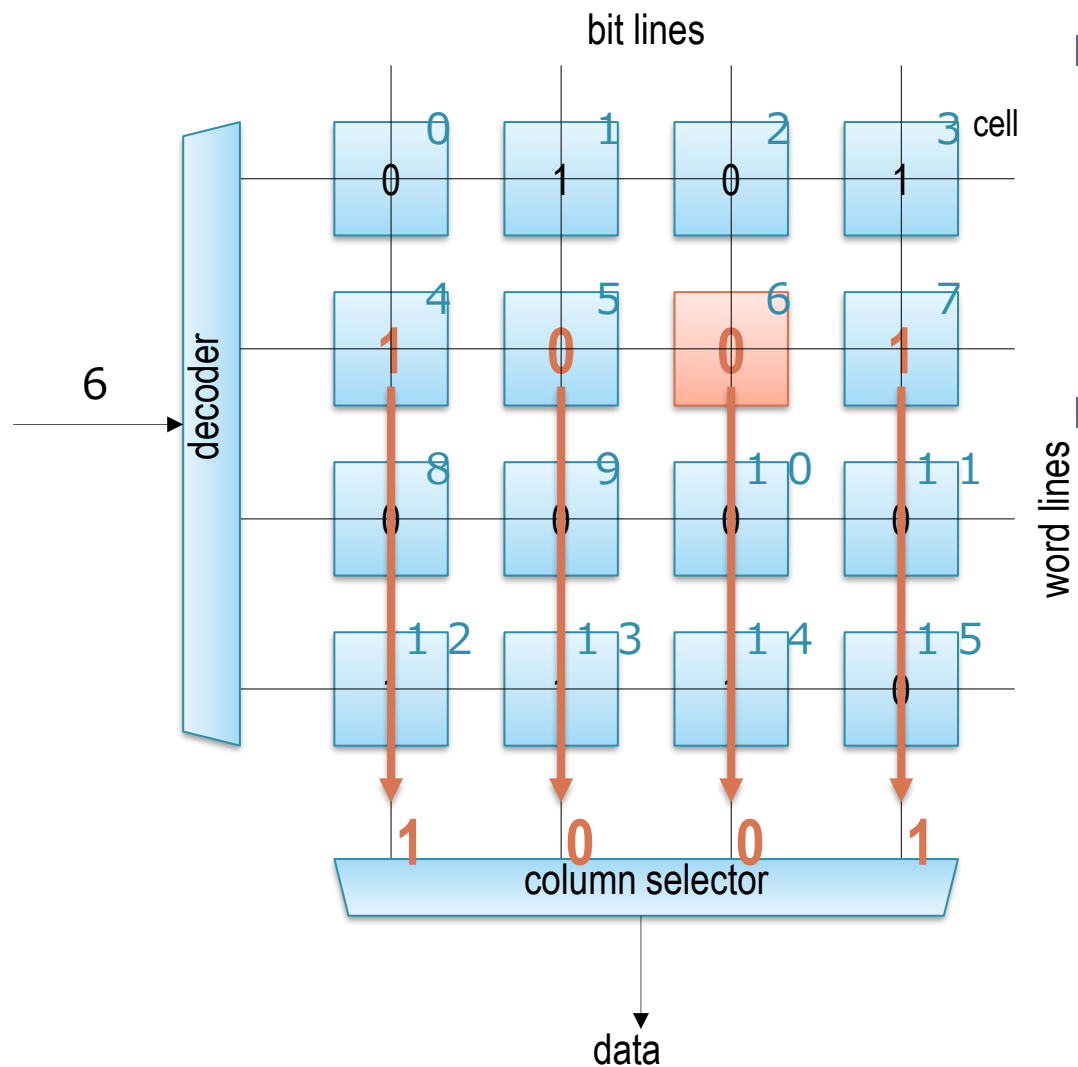
ワードラインのアサート



- 読み出す行に対応したワードラインをアサート
- 先ほどのデコード結果を使う

メモリの読み出し動作 (3)

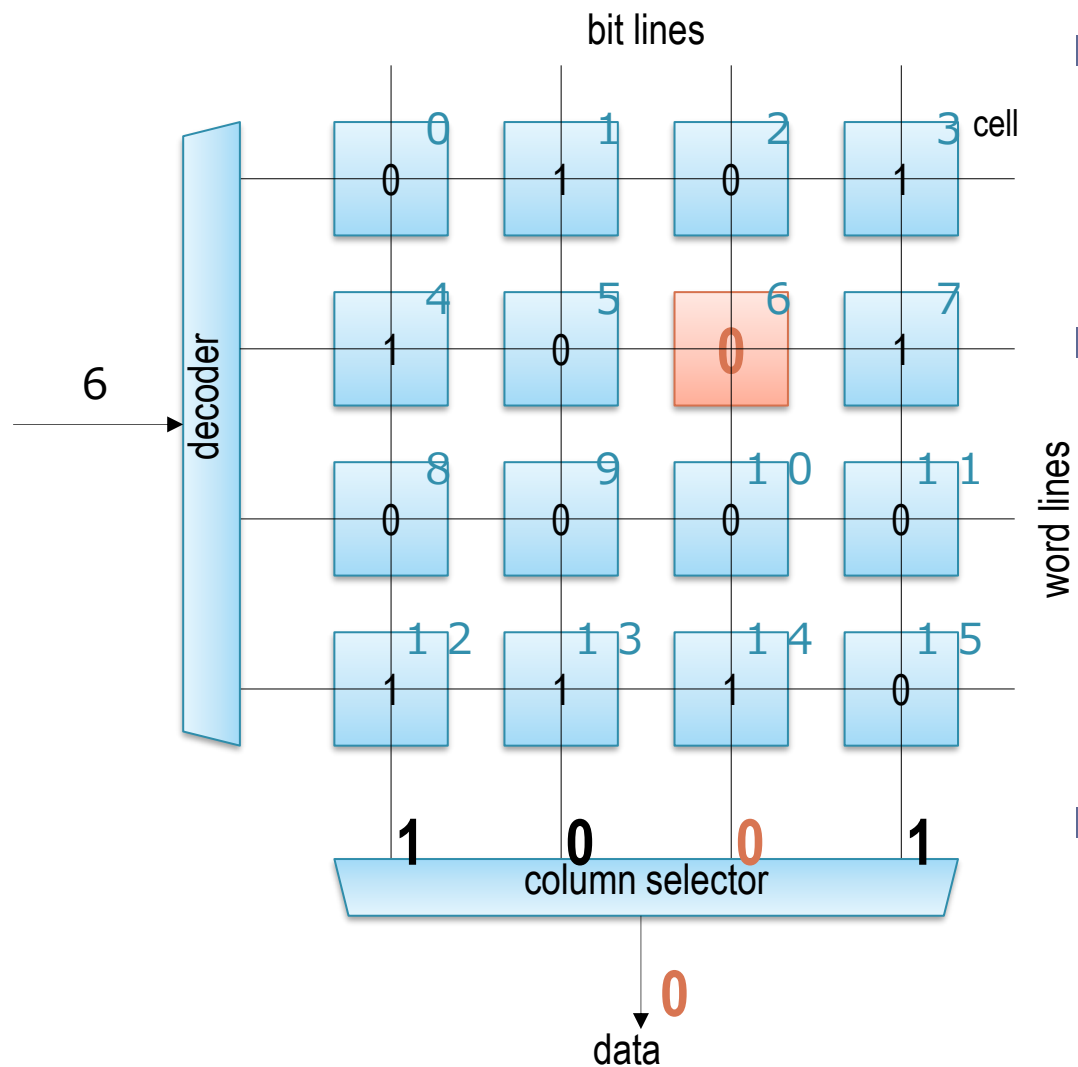
ワード (1 行分のデータ) の読み出し



- ワードラインがアサートされると, そこに接続されたセルがビットラインに自身の中身を流す
- これにより, 1 行分のデータが読み出される

メモリの読み出し動作 (4)

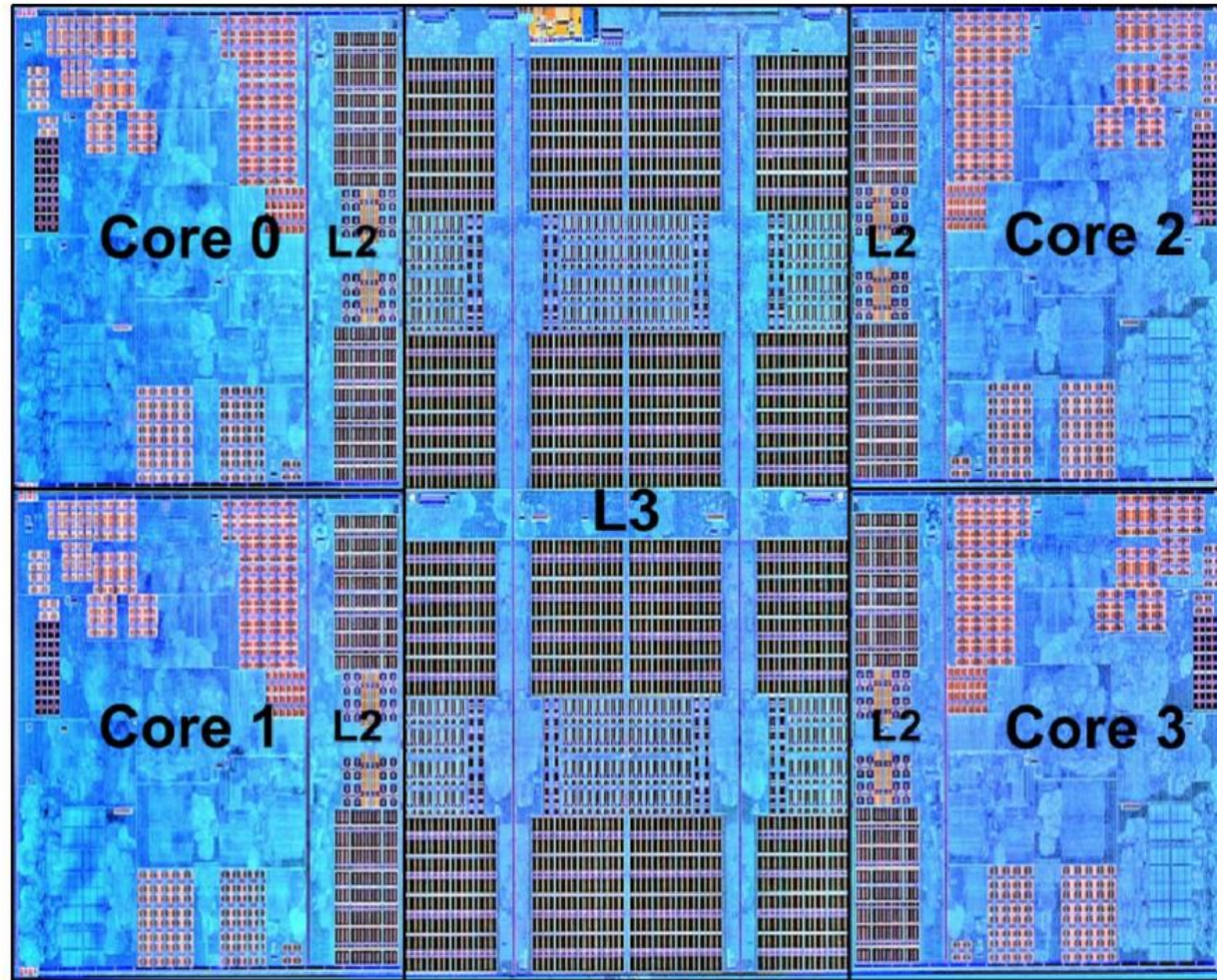
列の選択



- 読み出された 1 行分のデータから, 最終的に必要な 1 ビットを選択
- どの列を読むか決める
 - 各列は 4 つセルがある
 - アドレスを 4 で割った余りの列が読むべき場所
 - $6 = 0110$ (2 進数)
- カラム・セレクトによりビットを選択する
 - 読み出し幅によってはこの部分がないこともある

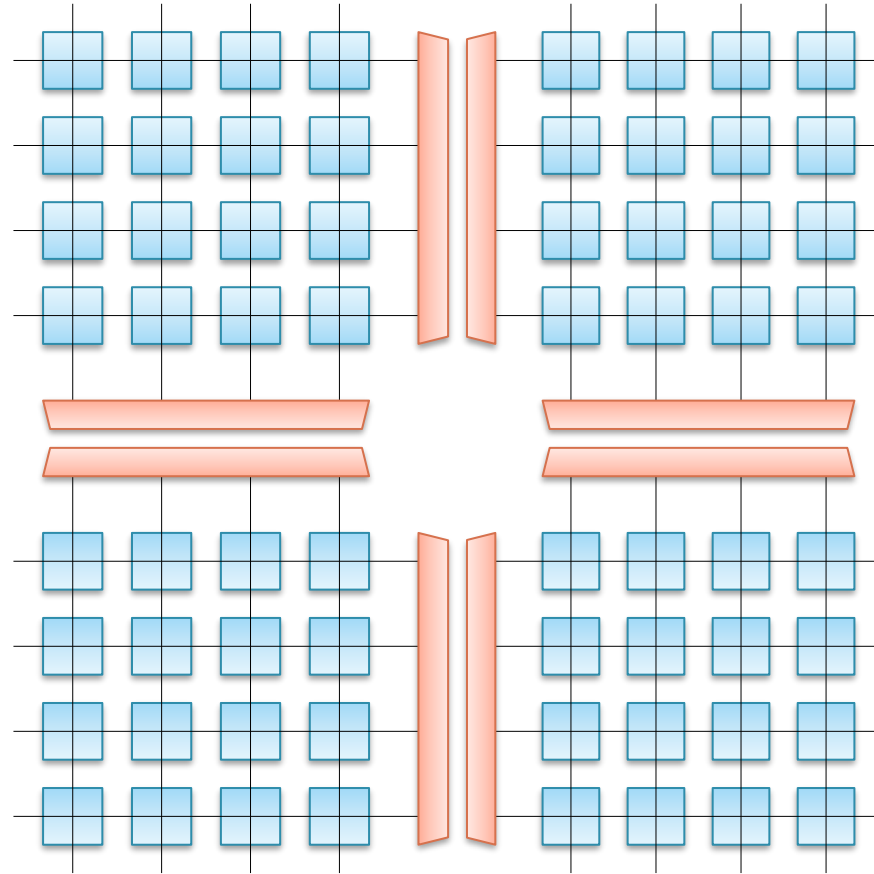
AMD Zen という CPU のチップ写真

Teja Singh et al., Zen: An Energy-Efficient High-Performance x86 Core より



- 赤や紫の「田」の字の構造の部分が全部メモリ（SRAM）
- レジスタやキャッシュ，各種テーブルなど

「田」の字の構造



- 「田」の字の構造になっているのは,
 - 縦線がデコーダで左右に対してワードラインをアサート
 - 横線でビットラインのデータを拾う

メモリ

1. メモリの基本

1. 構造
2. 動作
- 3. 容量と速度**

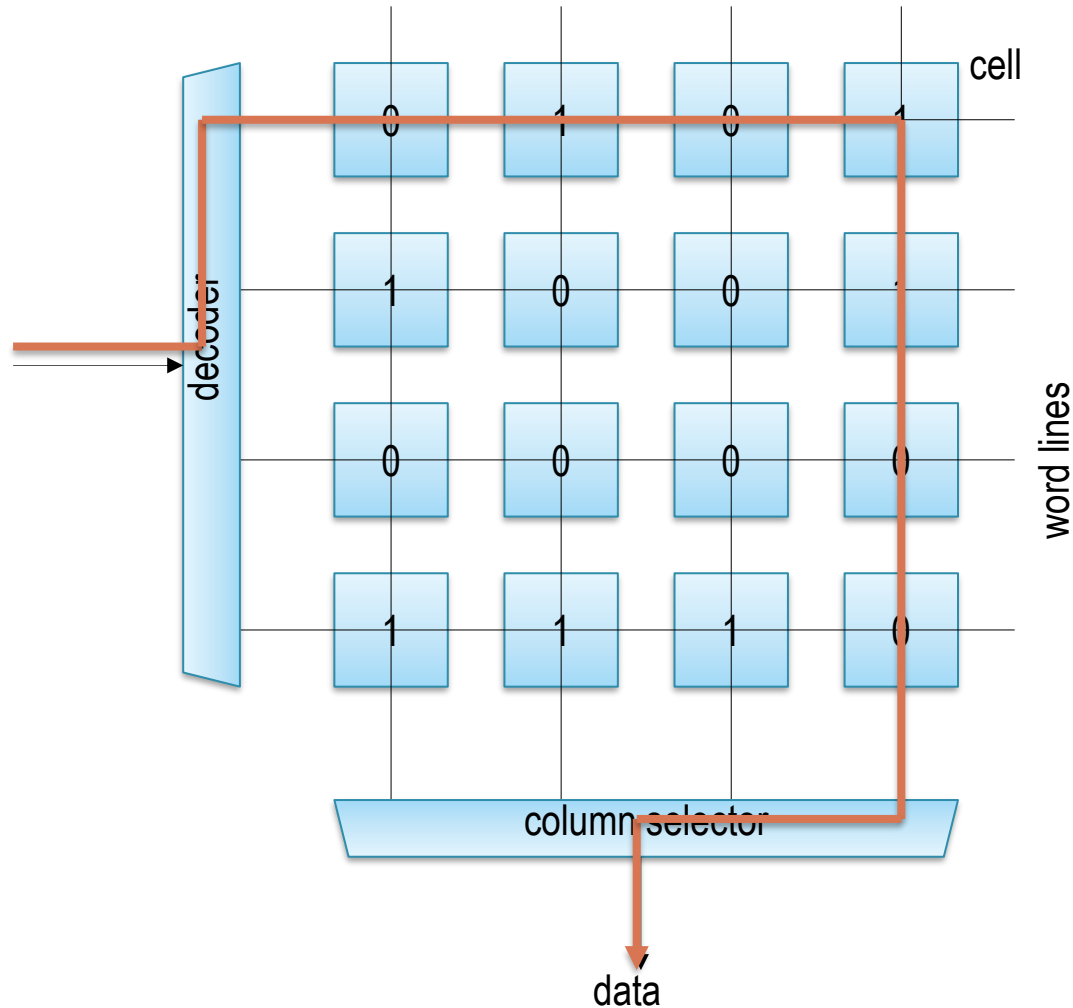
2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？
3. マルチポート・メモリ
4. メモリを対象にしたアタック

メモリの性質

- 容量（大きい or 小さい）
 - どのくらい数のデータを覚えることができるか
- 速度（速い or 遅い）
 - どのくらいの速さで読み書きできるか
- 容量と速度にはトレードオフがある
 - 「大きくて速くいメモリ」は作ることができない

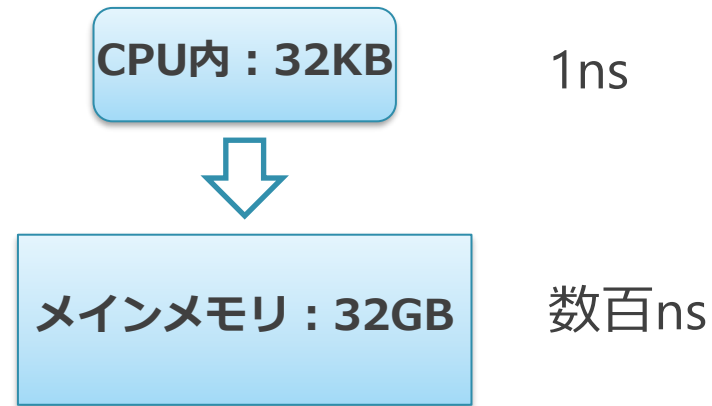
アクセス時間は容量の平方根ぐらいに比例



- 格子状に並んだセルの**外周部の信号線の長さ**がアクセス時間を決める
- 容量（セルの数）が一定の場合、正方形に近くするのが最も経路が短くなる
- **メモリ容量**
 $= (\text{外周部の長さ})^2$

アクセス時間は容量に直接は比例しない

(容量の平方根ぐらいに比例)



- 格子状に並んだセルの外周部の長さがアクセス時間を決めるから
 - メモリ容量 = (外周部の長さ)²
 - アクセス時間は、容量に対して線形には伸びない

速度

- 信号線の長さがなぜ問題に？
 - 電気信号が伝わる速度はとても速いのでは？
- 計算してみる：
 - 光の速度： 秒速 30万Km = 3×10^8 m
 - CPU の動作周波数： 3GHz = 3×10^9 Hz
 - 1回の処理で光が進める長さ = $(3 \times 10^8) / (3 \times 10^9) = 0.1\text{m} = 10\text{cm}$
- 光の速度でも大して進めないぐらい、今のコンピュータは速い
 - 回路中の電気信号の伝達は光よりもだいぶ遅い

メモリの速度と容量のまとめ

- 速さと大きさにはトレードオフがある
 - 「大きくて遅いメモリ」か「小さくて速いメモリ」しか作れない
 - 容量を大きくすると、その分メモリ内の信号線が伸びる
- 実際には、セルをどのような方式で作るかでも大きく変わる
 - しかし、上記のトレードオフは常になりたつ

メモリ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

2. メモリの詳細

1. SRAM と DRAM
2. なぜメモリが存在するのか？

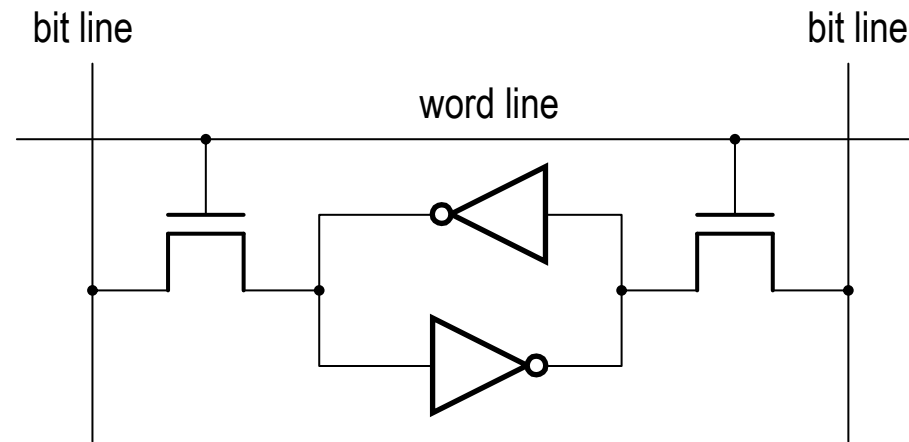
メモリのバリエーション

- メモリは基本的にはみな同じ構造を取る
 - 情報を記憶するセルの中身が方式ごとに異なる
- SRAM と DRAM を簡単に紹介
 - 現代のコンピュータはおおよそこの2種類で作られている
 - この講義的には性質の大ざっぱな違いがわかっているだけでよい
 - (細かい動作まではあまりわかってなくても良い)

Static Random Access Memory (SRAM)

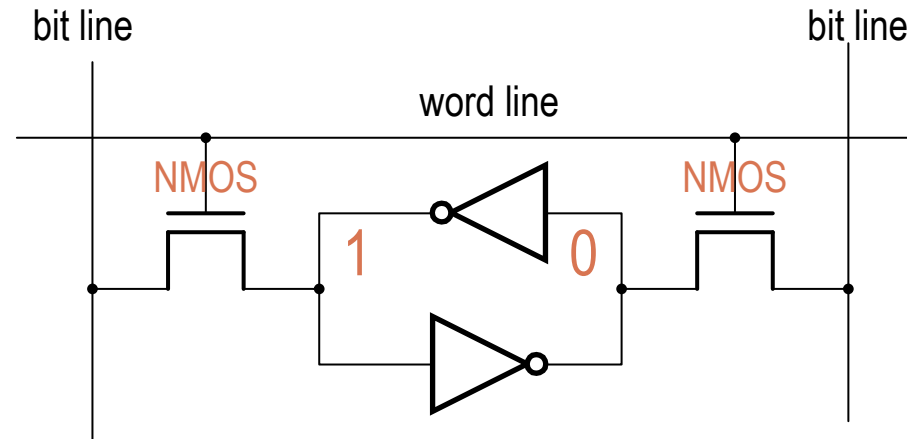
- セルにインバータ（NOT ゲート）のループを使った方式
- Static とはどういう意味か？
 - 動作が静的（何もしなければ電流をほとんど消費しない）
 - 電源を入れている限りは内容が消えない
- 一般的にはメモリの中で最も高速
 - CPU 内の論理回路と同じトランジスタを使って作るから
 - レジスタなどは SRAM で作られている

SRAM のセル (1bit)



- インバータのループにより 1 bit を記録
 - ループの左右が [1:0] あるいは [0:1] の 2 つの定常状態を持つ
- D-FF とかと同じ原理
 - ただし、クロックに同期させるための仕組みがない分単純

SRAM の読み出し

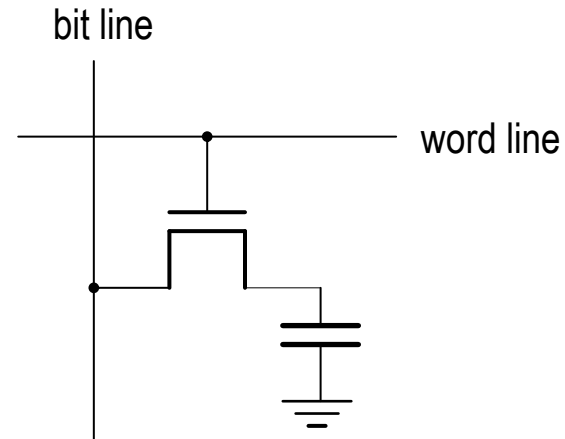


1. 読み出す行のワードラインが高電位（1）に
2. その行の NMOS が ON になりビットラインとインバータが接続
3. ビットラインを通じてセルの内容が伝えられる

Dynamic Random Access Memory (DRAM)

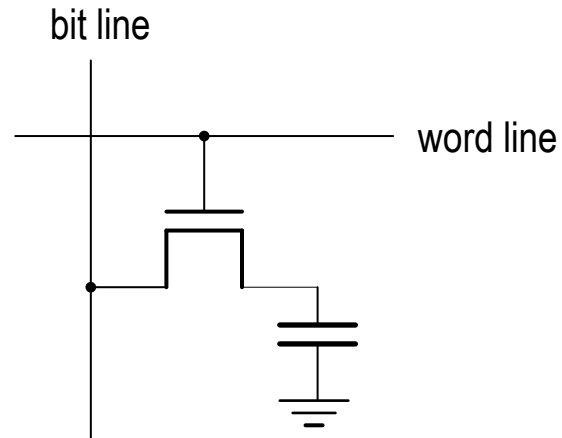
- セルにコンデンサを使った方式
- Dynamic とはどのような意味か？
 - 動作が動的：電荷がもれて記憶が消えていく
- 速度は遅いが、セルを小さく作れるので容量が稼げる
 - メイン・メモリは DRAM で出来ているのが普通

DRAM のセル



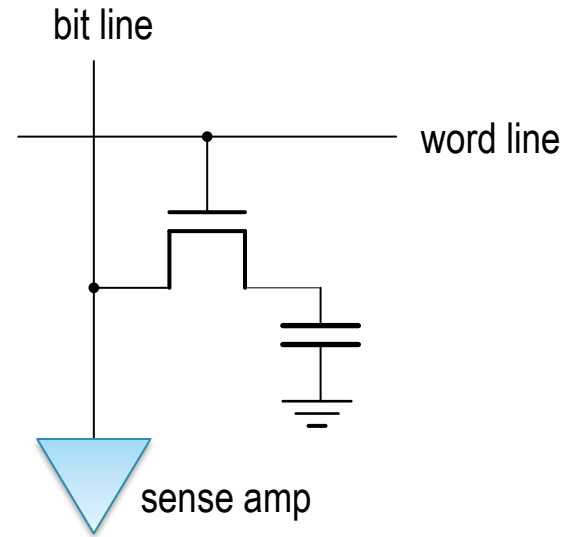
- セルをコンデンサによって構成
 - 電荷がたまっていれば 1 , そうでなければ 0
 - 時間が経つと自然に電荷が抜けてデータが消える
- 電荷を維持するため, 定期的にはリフレッシュと呼ばれる操作を行う
 - 具体的には, 一回読んで同じデータを書き戻す

DRAM の読み出し



1. ビットラインに電荷をプリチャージ
 - ビットライン全体を一種のコンデンサだと思う
 - 高電位にして電荷を信号線にためる
2. 読み出す行のワードラインを高電位に
 - NMOS が ON になりビットラインと接続
3. セルの状態に応じてディスチャージが起きる
 - セルに電荷がある（1）→ ビットラインの状態は変わらない
 - セルに電荷がない（0）→ ビットラインの電位が下がる

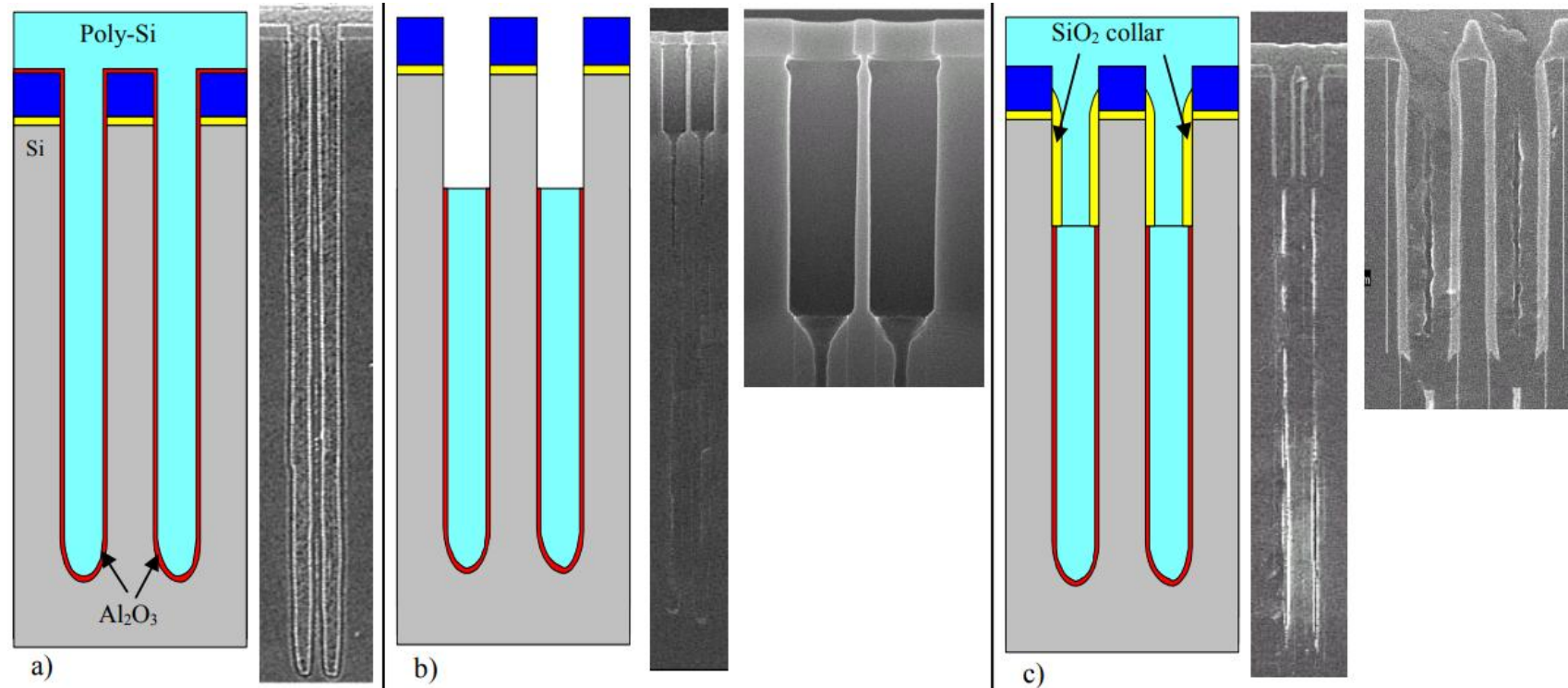
SRAM の場合との違い



- 読み出し時に内容が破壊される
 - ビットラインとコンデンサが接続されると、電荷が流れ出す
- コンデンサがビットラインをディスチャージする速度は遅い
 - センスアンプと呼ばれる微小な電圧の変化を増幅する回路を使う

DRAM セルのコンデンサの作り方

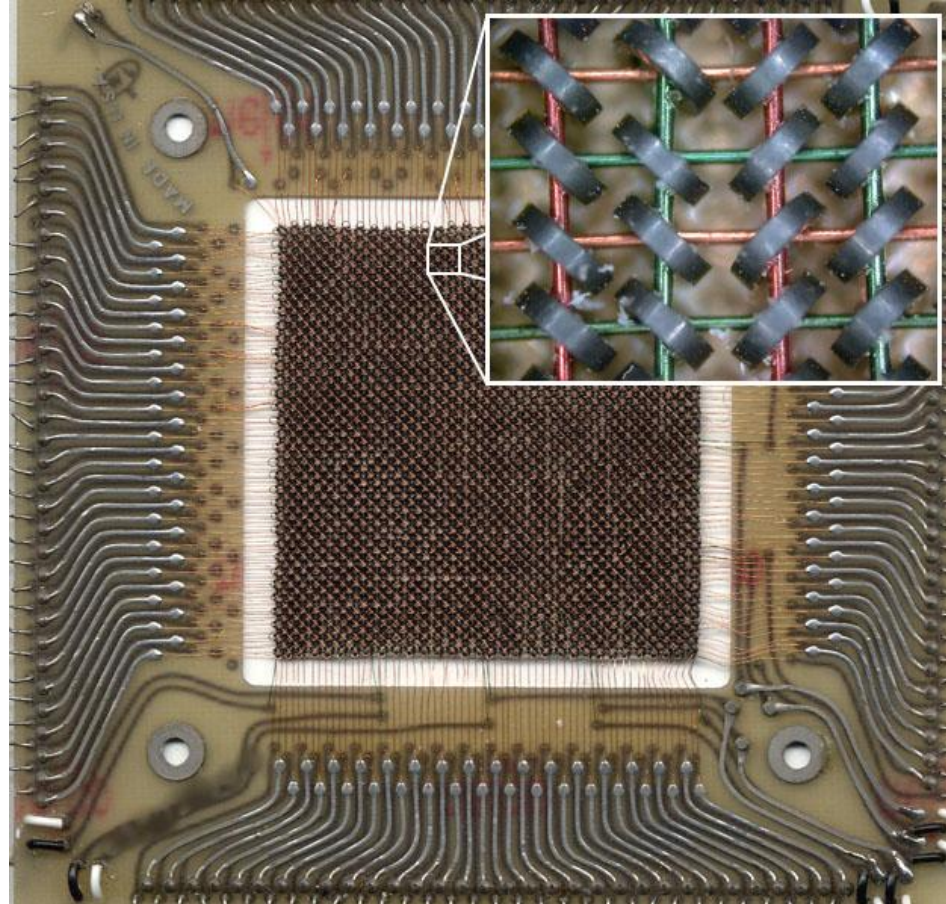
H. Seidl et al, A Fully Integrated Al₂O₃ Trench Capacitor DRAM for Sub-100nm Technology より



- チップの表から裏に向けて深い穴を掘って表面積を稼ぐ
 - コンデンサの容量は表面積に比例
 - トレンチ（塹壕の意味）と呼ばれる
 - 特殊な工程が必要なので通常のトランジスタと混ぜて作るのが難しい
 - DRAM は CPU とは違う専用のチップで製造される

過去に使われていたもの：磁気コアメモリ

<https://ja.wikipedia.org/wiki/%E7%A3%81%E6%B0%97%E3%82%B3%E3%82%A2%E3%83%A1%E3%83%A2%E3%83%AA> より



- 1950年代から使用されていた
- いわゆる「コアダンプ」のコアとは元々この磁気コアのこと

メモリ

1. メモリの基本

1. 構造
2. 動作
3. アクセス時間

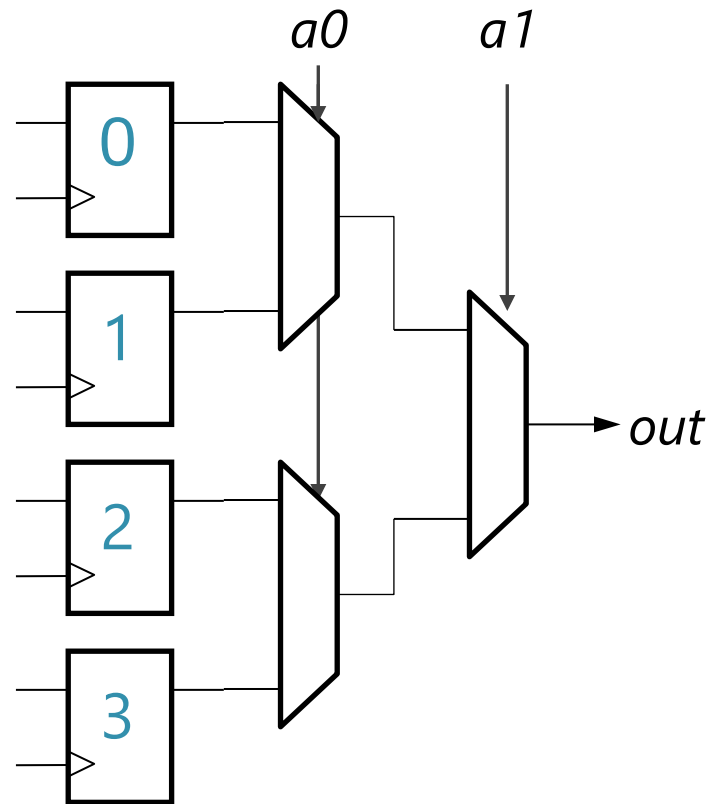
2. メモリの詳細

1. SRAM と DRAM
2. **なぜメモリが存在するのか？**

メモリの存在する理由

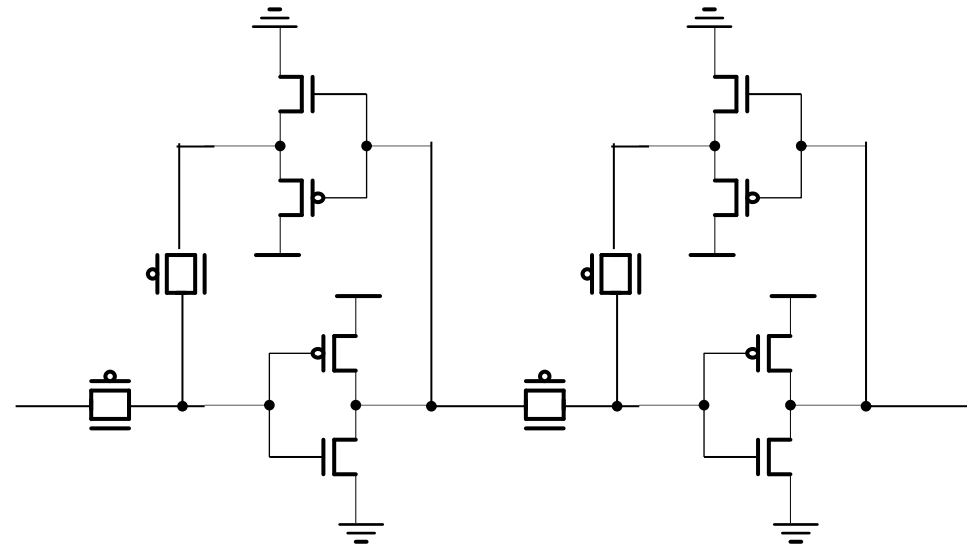
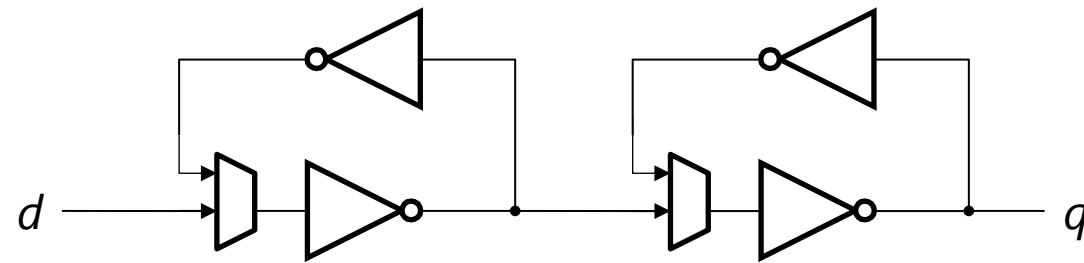
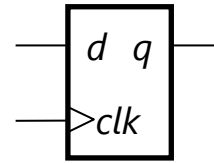
- D-FF とマルチプレクサがあれば、等価な機能は実現できる
 - しかし実際には、メモリ専用の回路が用意される
- メモリ専用の回路が用意される理由：
 - D-FF とマルチプレクサよりも、高密度に実装したいから
 - 同じチップ面積で、より多くの情報を記憶したい

D-FF とマルチプレクサによる 4bit の RAM

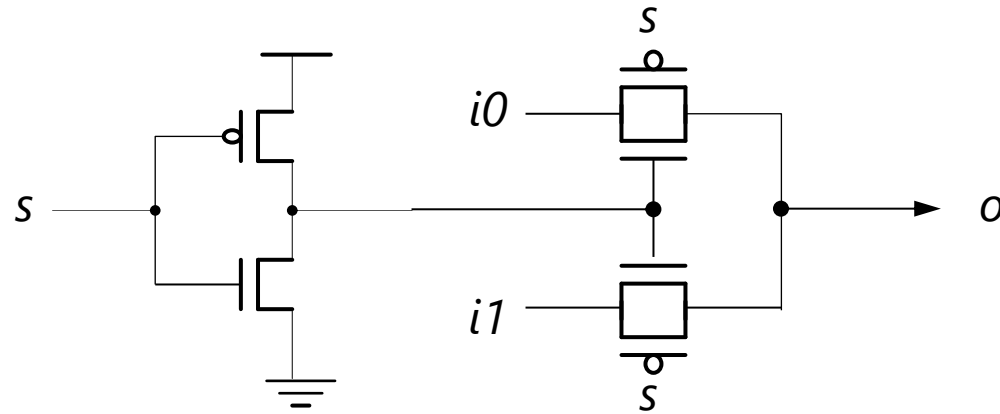
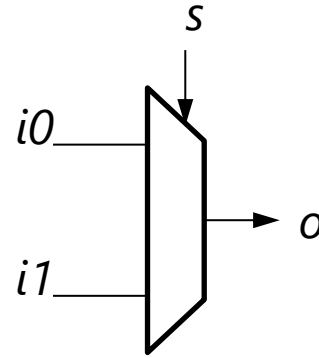


- 4 エントリの LUT を D-FF で構成してみる
 - 中身を憶える 4つの D-FF
 - 場所を指定して選択する2段のマルチプレクサ

D-FF : トランジスタ 16個

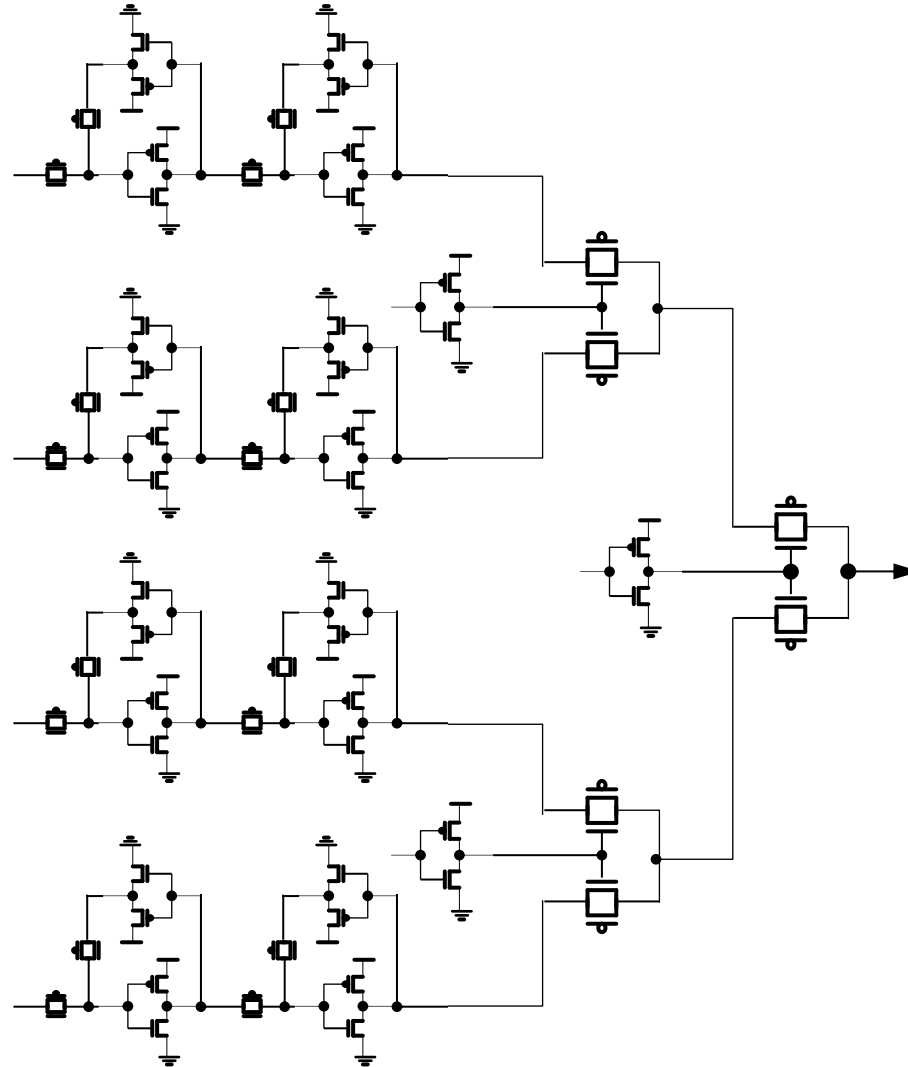


マルチプレクサ：トランジスタ 6個



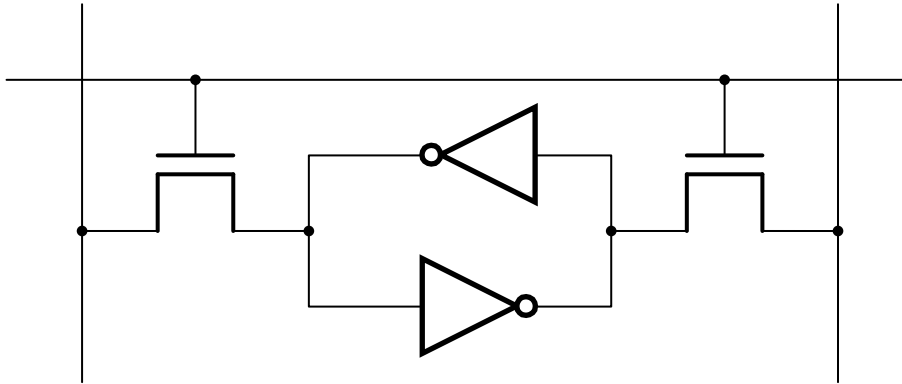
4bit メモリに必要なトランジスタ数 : 82

$$16 \cdot 4 + 6 \cdot 3 = 82$$

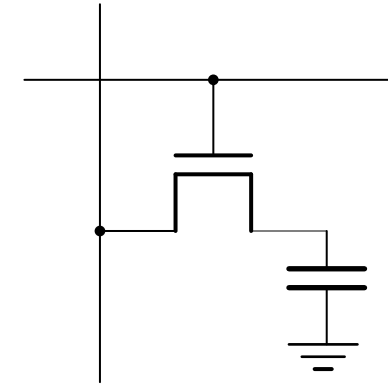


SRAM と DRAM のセル (1bit)

SRAM : $2 + 2 \times 2 = 6$



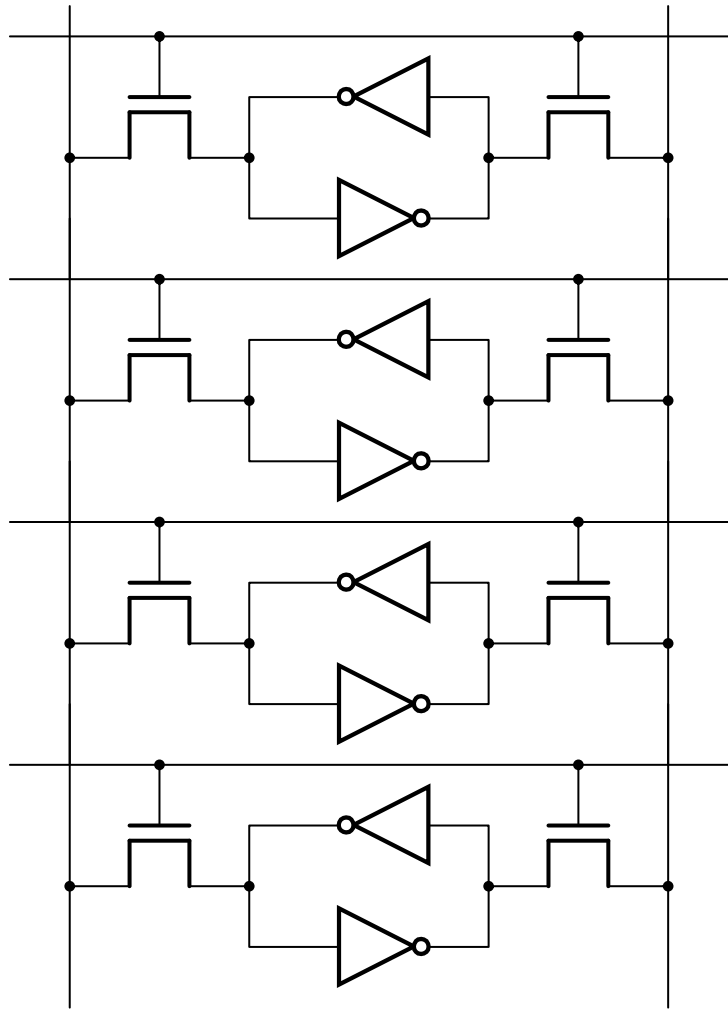
DRAM : 1 + コンデンサ 1



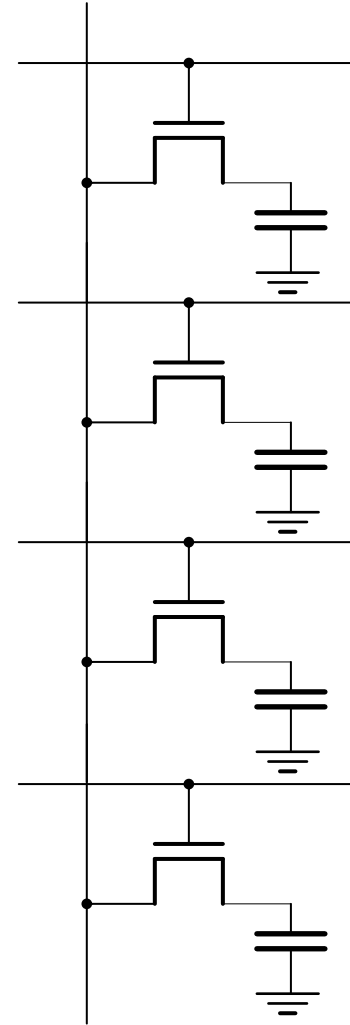
- D-FF よりも圧倒的に単純

SRAM と DRAM (4bit)

SRAM : $6 \times 4 = 24$



DRAM : $4 + \text{コンデンサ } 4$



メモリのまとめ

(おおよそこのぐらいの事がフワッと分かれば良い)

- メモリ (RAM: Random Access Memory)
 - 複数のデータを記憶し, 配列のように位置を指定して読み書きする
 - なるべく少ない回路で なるべく多くの記憶を行うために存在
- 構造とアクセス時間 :
 - 1bit の記憶を行うセルを格子状にならべた構造を持つ
 - 読み書きにかかる時間 :
 - 記憶容量の平方根程度に比例
 - 信号線の長さに比例するため, 速度と容量にはトレードオフがある

メモリのまとめ

(おおよそこのぐらいの事がフワッと分かれば良い)

■ SRAM :

- セルにインバータのループを使ったもの
- 高速だが低密度
- 電源を入れている限りは記憶が消えない

■ DRAM

- セルにコンデンサを使ったもの
- 低速だが高密度
- 電源を入れていても記憶が消えていく
 - リフレッシュが必要

キャッシュ

キャッシュとは？

- 「キャッシュ」って、ブラウザのあれ？
- ちょっと違うけど、原理は同じもの

原理は同じ

■ ブラウザのキャッシュ

- WEB サーバーからページを取ってくるのは遅い
- 1回見たページを PC やスマホの「キャッシュ」に置いておく
- 2回目からは表示が速い

■ メモリのキャッシュ

- メイン・メモリからデータを取ってくるのは遅い
- 1回読んだデータを CPU の「キャッシュ」に置いておく
- 2回目からは読み込みが速い

性能へ大きく影響するし、影響範囲も広い

- キャッシュは現代のコンピュータにおいてとても重要
 - ほとんどのプログラムで性能に大きな影響を与えている

例：行列積の実装と性能

- 三重ループとして実装できる

- ループの順番は任意に入れ替え可能

```
for (int k = 0; k < SIZE; k++)  
    for (int j = 0; j < SIZE; j++)  
        for (int i = 0; i < SIZE; i++)  
            a[k][j] += b[k][i] * c[i][j];
```

- 単にループの順番を入れ替えるだけで処理時間が大きく変化する

- 外側から k j i の順 → 178秒
- 外側から k i j の順 → 20秒
- 外側から j i k の順 → 1100秒

- この変化は、キャッシュを有効に働かせているかどうかによって由来

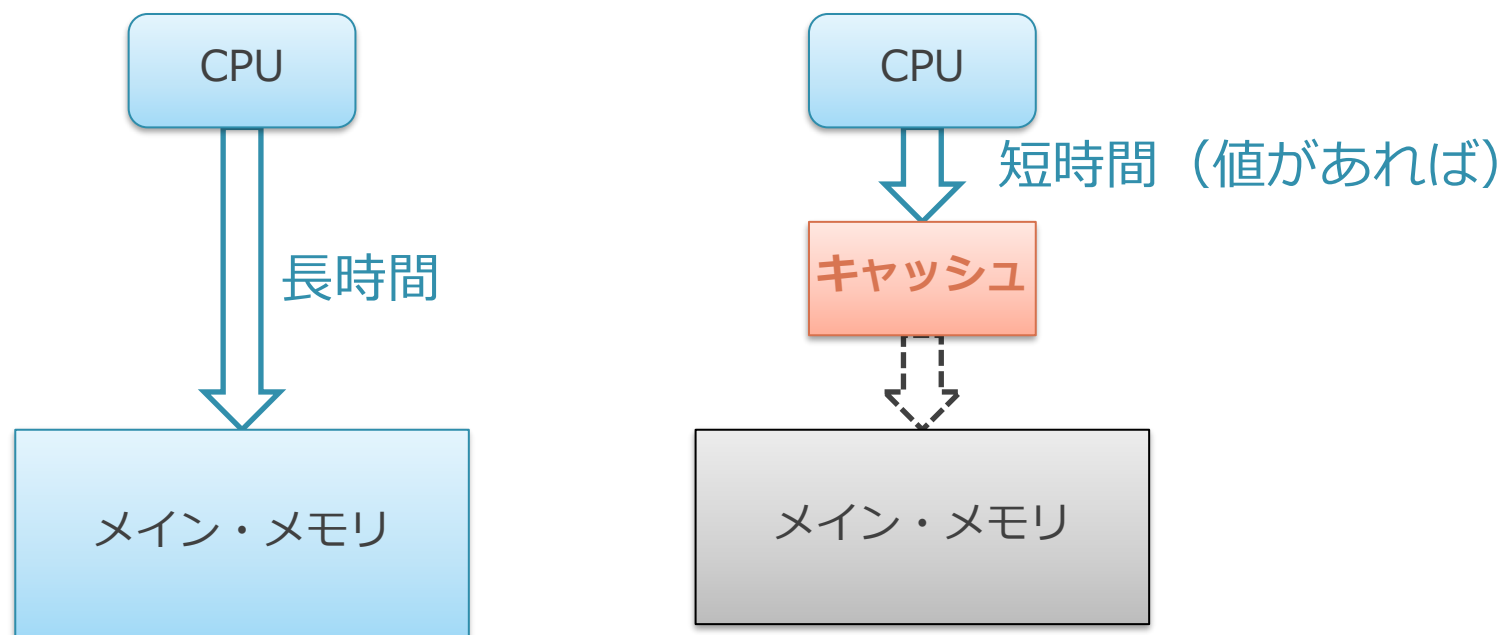
- 計算量や全体としてのメモリ使用量は全く同じなのに

1. キャッシュの基本的な考え方

1. 基本的な原理と構造
2. 容量の性能への影響

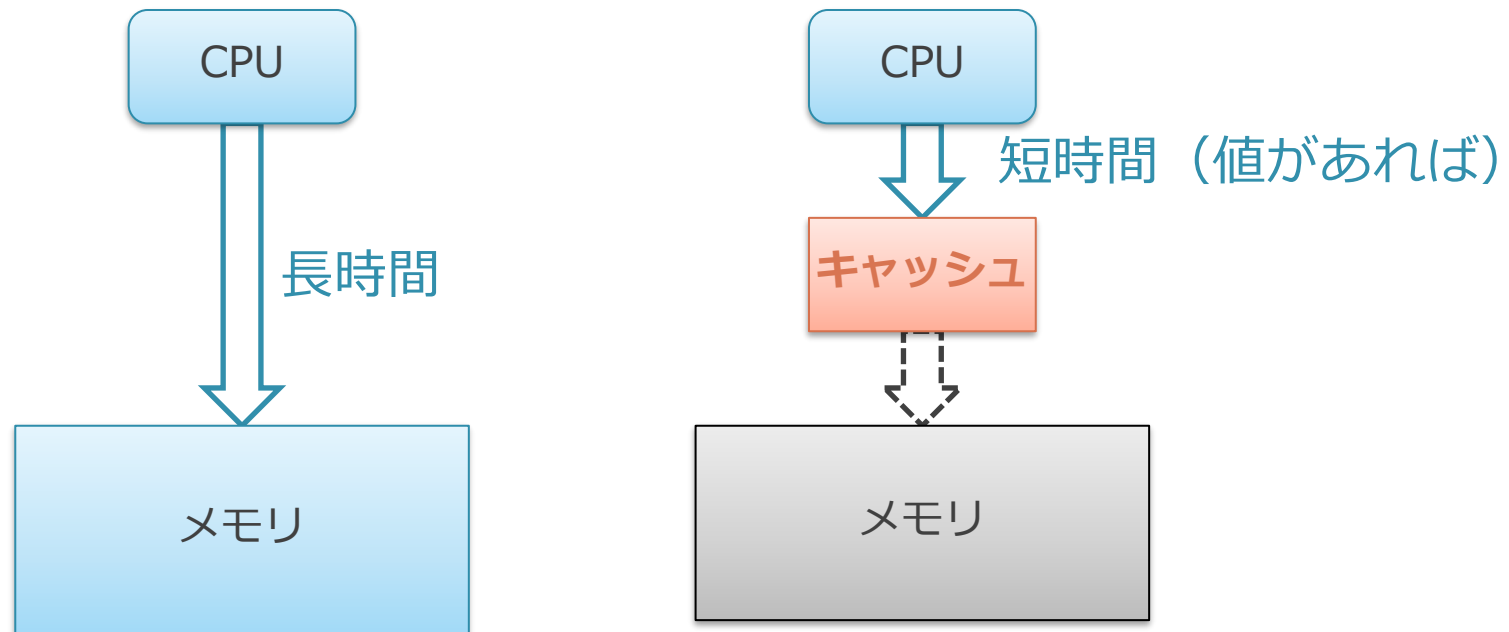
記憶階層

- 以下を階層的に組み合わせる
 - 小さいけど速いメモリ：キャッシュ
 - 大きいけど遅いメモリ：メイン・メモリ
- 背景：大きくて速いメモリは作れない



キャッシュの動作

- ソフトの性質：
 - 一度使用した値は，すぐにまた使う可能性が高い
 - ブラウザのキャッシュと同じ
- 一度利用した値を入れておくことで，2回目からは高速に

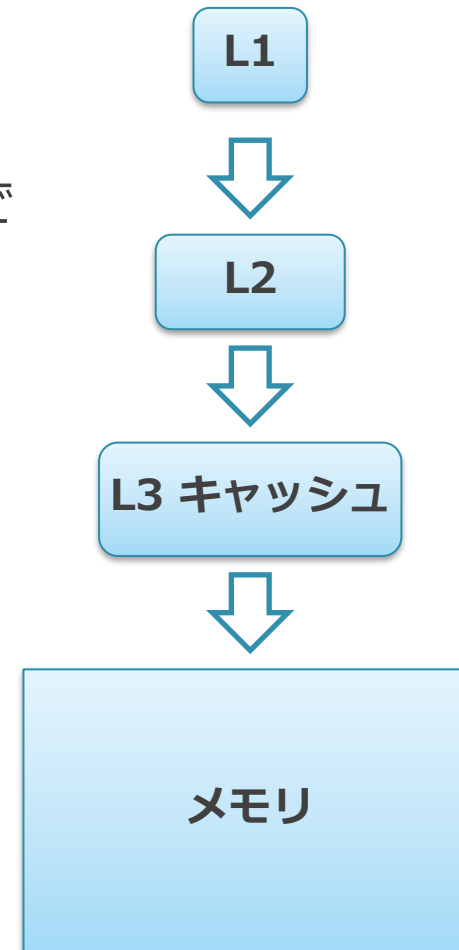


時間的局所性

- 「一度使用した値は、すぐにまた使う可能性が高い」という性質
- なぜか？
 - 人間は、「普通は」関連のある処理同士を近くを書く
 - プログラムの各行ごとに全く関係無い処理を混ぜたりは「普通は」しない
- 関連する処理は、当然同じデータを使う可能性が高い
 - 同じ入力データの使いまわし
 - 前の行の結果を次の行の入力で使う
 - for ループなどはこれらの典型

実際には多層の構造になっている

- 中間の速さと大きさのキャッシュが複数組み合わされている
 - PC に使われている CPU だと L1 ～ L3 まであるのが一般的
 - L は Level の略で、大きいほど（高次）CPU から遠い



キャッシュの基本的な考え方のまとめ

■ 記憶階層

- 小さいけど速いメモリ：キャッシュ
- 大きいけど遅いメモリ：メイン・メモリ

■ 時間的局所性

- 一度使用した値は、すぐにまた使う可能性が高い性質のこと

■ 実際には多層の構造になっている

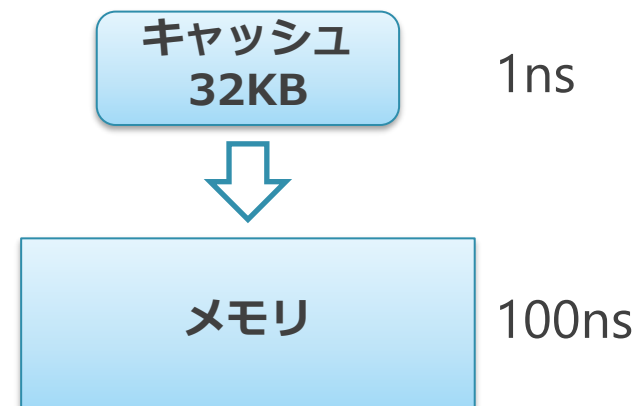
- 中間の速さと大きさのキャッシュが複数組み合わせられている

1. キャッシュの基本的な考え方
 1. 基本的な原理と構造
 2. **容量の性能への影響**

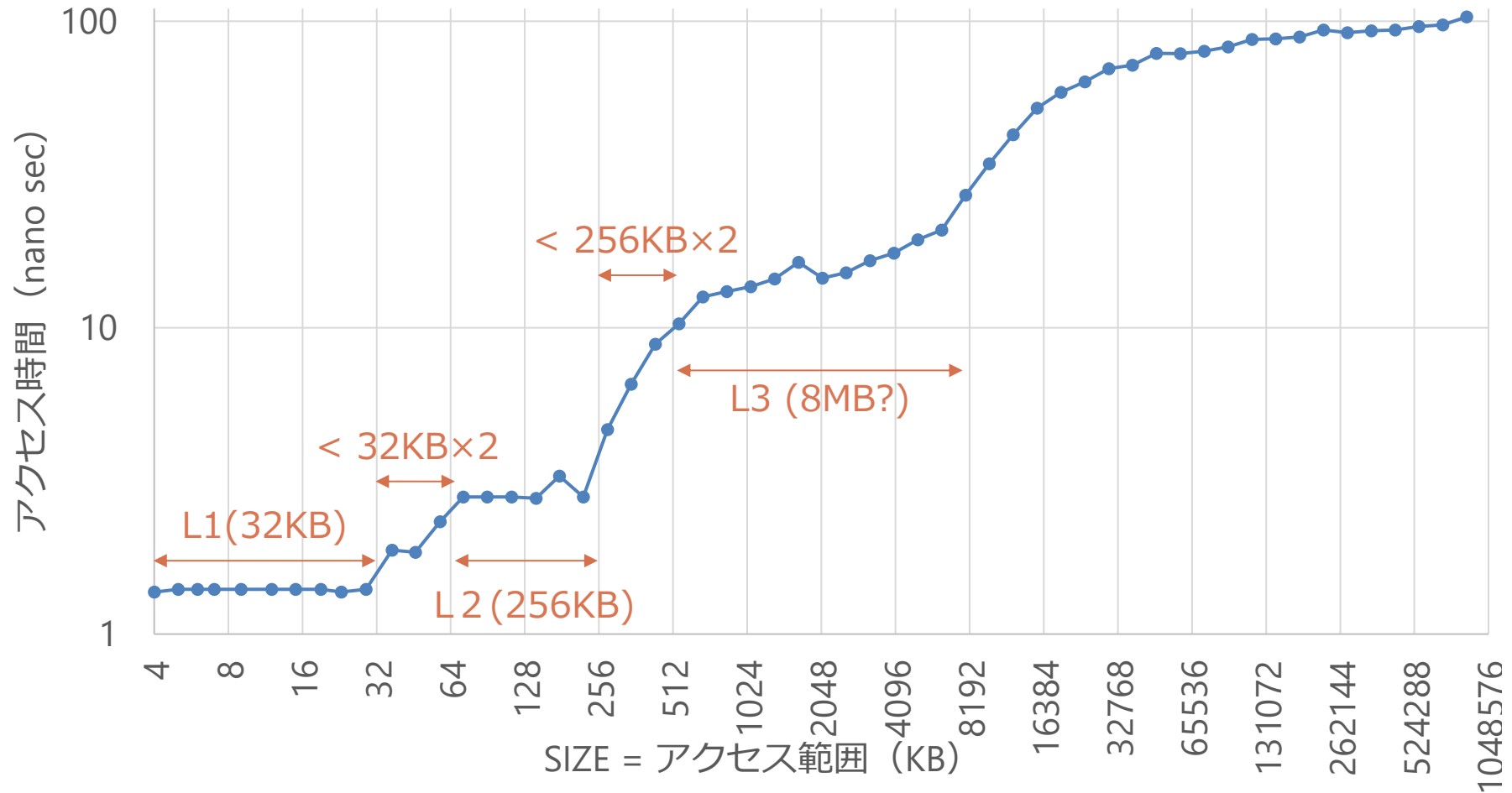
キャッシュへの性能への影響

- 下記のような二重ループを考える
 - SIZE がキャッシュの容量に収まっていれば、内側ループ終了後に table の全データがキャッシュに乗る
 - 次の内側の周回では全データがキャッシュに乗っているので速い！

```
for (int i = 0; i < NUM_TEST; i++) {  
    uint32_t p = 0;  
    for (int j = 0; j < SIZE; j++) {  
        p += table[j];  
    }  
}
```

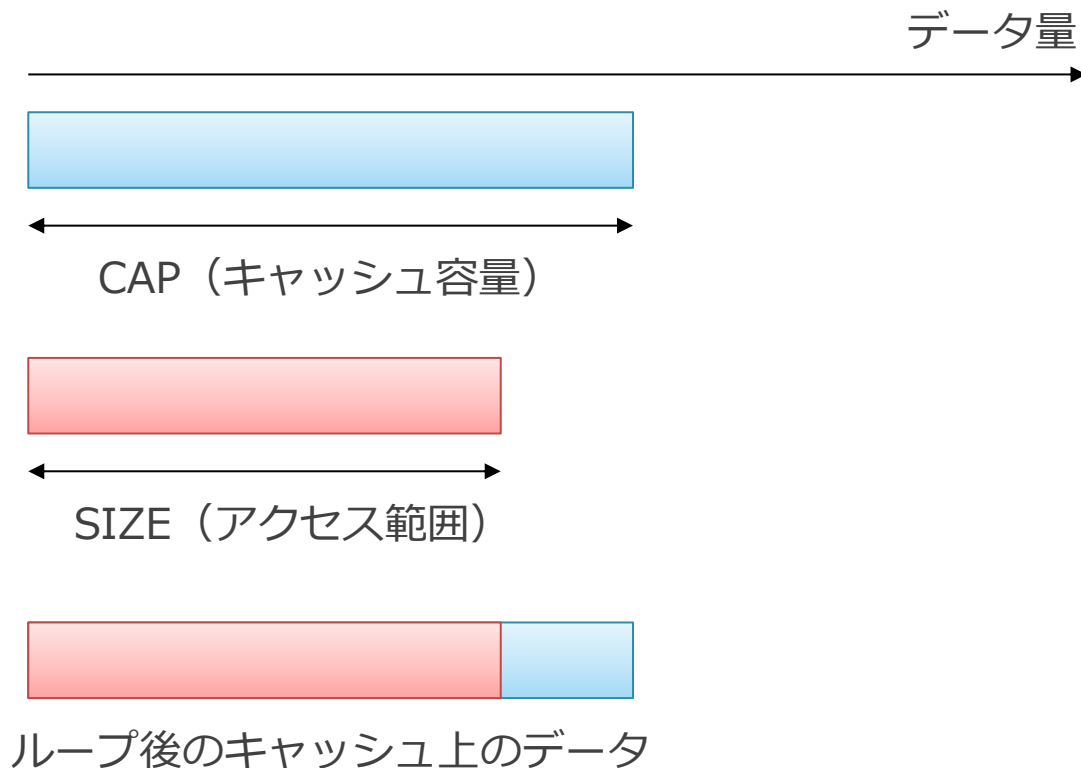


実際の測定データ



- 縦軸を基数10の対数軸, 横軸を基数2の対数軸に
- 低次キャッシュの内容は必ず高次に含まれる仕様
 - (そうなるかはメーカーや世代に依存. 含まれないこともある)
 - 256KB+32KB ではなく 256KB で変化しはじめる

プログラム最適化の上で、重要なポイント



■ 上記の状態を保つこと

- ワーキング・セット：
 - 処理のまとまりごとに、アクセスするデータの範囲（使用量）
- ワーキング・セットがキャッシュ容量に収まることが重要

まとめ

■ メモリ

- 構造, 動作, 容量と速度
- SRAM と DRAM
- メモリの存在理由

■ キャッシュ

- 基本的な原理と構造
- 容量の性能への影響

課題 9

■ 以下のような条件を考える

- 10段のパイプラインを持つ 2-way スーパースcalarプロセッサであり, 理想的には $IPC=2$ で実行できる
- ロード命令の出現率は 0.15
- CPU は L1 キャッシュをもつ
- ロード命令のみが L1 キャッシュやメモリにアクセスするものとする
- L1 キャッシュのヒット時には一切のペナルティなしで実行できる

■ (1) 以下の場合の IPC を求めよ

- ロード命令による L1 キャッシュのアクセス 1 回あたりのミス率が 0.02
- ミスの発生時は 100 サイクル追加で時間がかかる

課題 9

- (2) 以下の場合の IPC を求めよ
 1. L1 キャッシュの容量を倍にしたもの
 - L1 キャッシュに良く当たるようになったため、ミス率が 0.01 に
 2. L2 キャッシュを追加したもの
 - L2 へのアクセス 1 回あたりのミス率は 0.1
 - * L2 は L1 にミスしたときのみアクセスするものとする
 - L2 ヒット時は L1 ヒット時からの実行時間の追加が 15 サイクル
 - L2 ミス時は L1 ヒット時からの実行時間の追加が 150 サイクル
- (3) これまでに出てきた 3 つのモデルの性能を求め比較せよ
 - ただし L1 キャッシュ容量を倍にした場合、キャッシュのアクセス時間が伸びてしまうため、周波数が 0.8 倍になるものとする

提出方法

■ 以下を提出：

1. 課題 9：

- 提出は Moodle の「課題 9」のところからお願いします
- 紙に書いた場合は写真を撮ってアップロードしてください

2. 感想や質問：

- 「感想や質問」のところに投稿してください
- わからない場所がある場合、具体的に書いてもらえると良いです

■ 提出締め切り

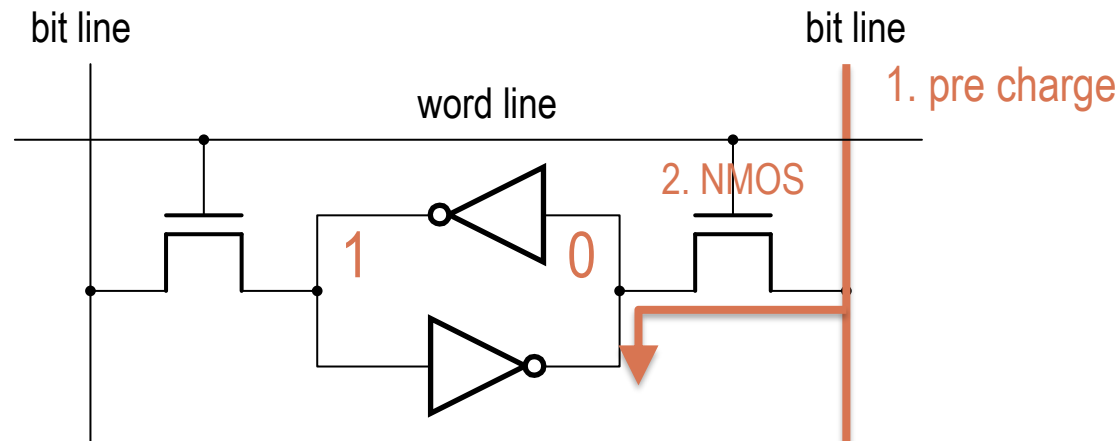
- 次の講義の開始まで

■ 注意：

- 課題の出来は、ある程度努力したあとがあれば良しです
- 必ずしも正解していなくても良いです

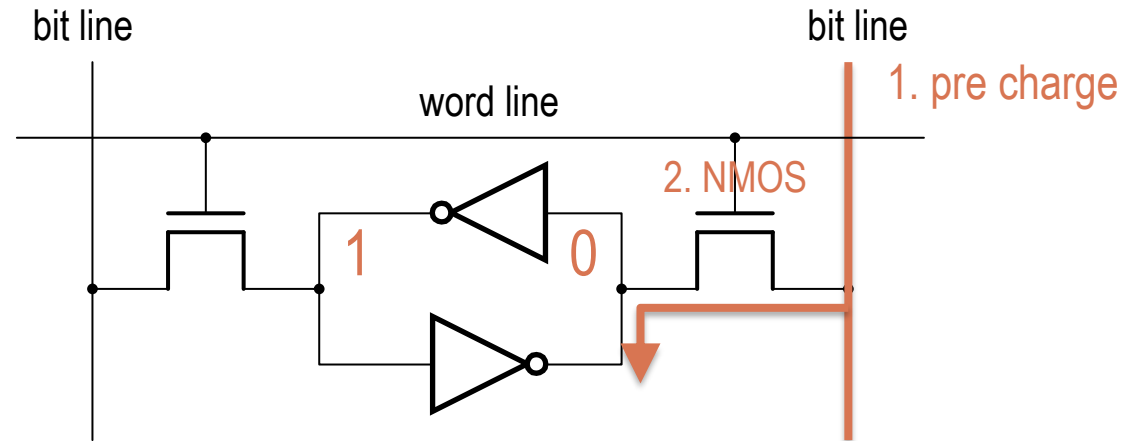
付録

SRAM の読み出し



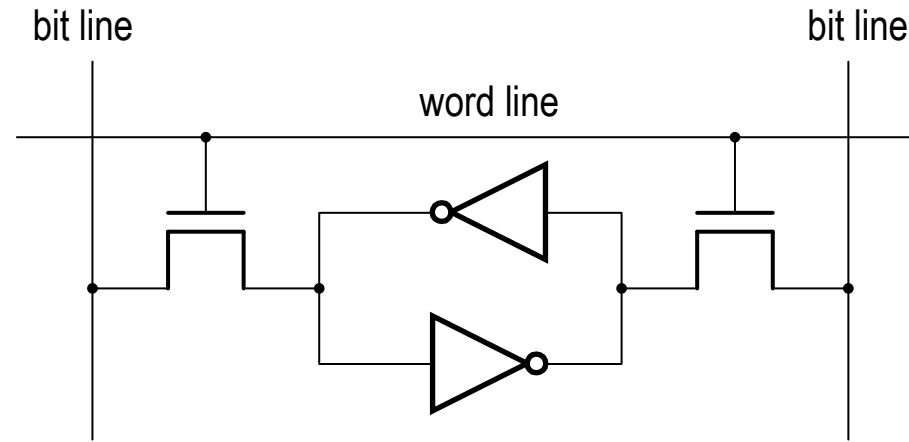
1. ビットラインのプリチャージ
 - ビットラインをあらかじめ高電位（1）にチャージする
2. ワードラインをアサート
 - ループの右にある NMOS が ON に
 - ループとビットラインが接続される
3. ビットラインのディスチャージ
 - ループの右が 0 なら, ビットラインが 0 に
 - ループの右が 1 ならそのまま

なぜプリチャージが必要なのか？



- 単にループとビットラインが接続されるだけではダメ
 - NMOS が高電位をうまく伝えられないから
 - もしループの右側が 1 で NMOS が ON になっても、ビットラインを 1 に引き上げることはできない
 - PMOS を追加すればできるが、なるべく小さく作りたいから嫌だ
- NMOS を介してビットラインの電位を下げるだけができる
 - = あらかじめ電位を上げておいて、下がったかどうかで判定

SRAM の書き込み

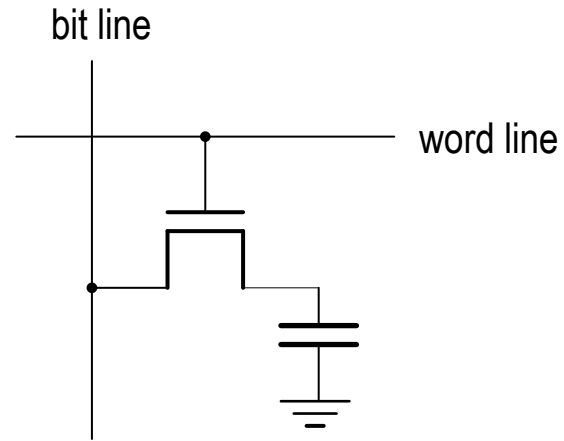


■ 書き込み手順

1. ループ左右のどちらか 0 にしたい方のビットラインの電位を下げる
2. ワードラインをアサートして NMOS を ON に
3. インバータの状態を強制的にビットライン側から低電位にする

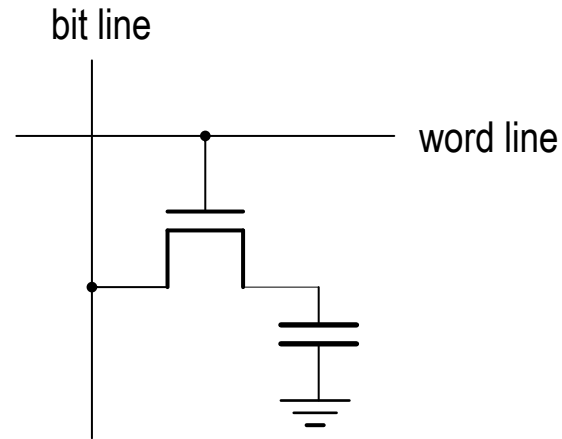
■ NMOS が低電位しか通せないなので、書き込みには 2 本いる

DRAM の読み出し



- 読み出しは, 基本的には SRAM と同じ
 - ビットラインをプリチャージ
 - ワードラインをアサート
 - セルの状態に応じてディスチャージが起きる

DRAM の書き込み



- 書き込みは, NMOS でも 0.5 までは上げられるのでそれで行う (多分)
 - SRAM ではインバータをひっくり返す必要があるので, 0.5 ではダメだった
- DRAM では 低電位 (0) or 中間電位 (0.5) を記録する
 - 読み出し時のプリチャージの電圧を 0.5 にして, 下がったかどうかで判定

余談：Branch Prediction Championship

Next-Generation Championship in Branch Prediction



Opening Remarks by
Aaron Lindsay, Organizing Chair
June 28, 2026

A Brief History of CBP

- 6 previous CBPs spanning 21(!) years
 - Valuable simulators, primarily measured MPKI with fixed storage budget
 - New, relevant traces to the community
- Amazing amount of effort by organizers and participants
- Inestimable contributions to state-of-the-art
- (Re-)invigorated branch prediction research

2004 Championship Branch Prediction Workshop Agenda and Results

Agenda: The slides are provided for each presentation. For each predictor, the writeup and code is provided. The code is provided as a uuencoded bziped tar file (.tar.bz2.uue) formatted according to the directions found by clicking [here](#). The traces are not provided with the code, but are provided separately as part of the CBP framework. The CBP framework can be downloaded from the following [page](#). The predictors require the traces with data values and memory addresses, so you don't need those traces.

0:00 Introduction: Competition Overview and Workshop
slides ([ppt](#) [pdf](#))

0:00 Perceptrons for Dummies
slides ([ppt](#) [pdf](#))

**Championship Branch Prediction Program
Sunday, June 15, 2014**

Authors	Paper	Code	Presentation
Introduction Areshi (Georgia Tech)			slides
Local Predictor Lipasti (University of Wisconsin, USA)	pdf	tarball	slides
Hashing and Mikko Lipasti (University of Wisconsin, USA)	pdf	tarball	slides
Sampling Hashed Perceptron Predictor Jimenez (Texas A&M University, USA)	pdf	tarball	slides
Branch Prediction using Multi-dimensional Histories Jencio, Joshua San Miguel, Natalie Enright Jerger, and Andreas Moshovos (University of Toronto, Canada)	pdf	tarball	slides
Local Stream Branch Predictor Gaming Shen, Michael Ferdman (Stony Brook University, USA)	pdf	tarball	slides
Pattern: A Hybrid Pattern Based Dynamic Branch Predictor with Performance Based Kaushik Garikipati, Milan Patnaik, V. Kamakoti (IIT Madras, India)	pdf	tarball	slides
=====BREAK=====			

NC STATE UNIVERSITY

CBP2025 Workshop

Saturday, June 21, 2025, morning
Location: Room 26-B1F (Building 2)

9:00 – 9:20: Opening Remarks. Rami Sh.

9:20 – 10:50: Session 1

9:20 – 9:35: TAGE-SC for CBP2025. André Seznec
(University of Murcia).
[\[presentation\]](#)[\[youtube\]](#)[\[paper\]](#)[\[code\]](#)

9:35 – 9:50: A Deep Dive Into TAGE
[\[presentation\]](#)[\[youtube\]](#)

9:50

CBP-NG Motivation

Does it produce a prediction fast enough?

How do you pipeline it?

Does it support the needed throughput?

Does it use too much energy?

How do you (efficiently) train arrays without interfering with prediction?

Do throughput, energy, and/or delay issues warrant a new approach?

First Place Goes To...

**Toru Koizumi, Masanari Mizuno, Soma Nishida,
Kanata Abe, Tomoaki Tsumura, and Ryota Shioya**

for their entry titled

*MORSL: Minimal-Overhead Rank-Based Predictor
with Summation-Free Correction and Lazy Access*

(VFS of 0.9957)

トロフィー



Second Place Goes To...

Jun Fan

for their entry titled

*Enhance Ahead-Pipelined N-Branch
GShare with Tagged Tables*

(VFS of 0.9948)

Acknowledgements

Organizing Committee

- Aaron Lindsay (Ampere), Chair
- Pierre Michaud (Inria)
- Rami Sheikh (ARM)
- Mahesh Madhav (Ampere)
- Alaa Alameldeen (Simon Fraser)

Program Committee

- Pierre Michaud (Inria), Chair
- Alaa Alameldeen (Simon Fraser)
- Daniel A. Jiménez (Texas A&M)
- Eric Rotenberg (NC State)
- Gilles Pokam (AMD)
- Houdhaifa Bouzguarrou (ARM)
- Michael Chin (Ampere)
- Rami Sheikh (ARM)
- Syed Fakhri (Ampere)
- Thibaut Lanois (ARM)

質問とか感想

質問とか感想

- 前回、前々回もその前もずっと出席していた（証明のしようがないのですが）のですが、本日どうしても体調が優れず休んでしまいました。資料を見て勉強していたところ、出席調査が突然出てきて驚いたのですが、成績に反映されますか？
- 体調を崩して前回欠席してしまったのですが、出席が取られていたと聞いて少し不安です 😞
1回の欠席で落単になる可能性はありますか。

質問とか感想

- 最近は工場のベルトコンベアに例えられた言葉での説明が多かったけれども、今日の授業はモデル式などがたくさん出てきて難しかった。

質問とか感想

- 今までパイプラインについてなんとなく理解しているがしっくりはきていないという感じだった。今回の授業で式を用いてその効率が実際にどう変わるのか明確になったので一気に理解できた。言葉での理論の説明より、実際に値を仮定したモデルで計算して、「ほら値増えてるでしょ」の方が、自分にはあっているのだと思った。

質問とか感想

- パイプライン段数を2倍にするとどうなるか、といった内容が資料に書かれていたが、パイプライン段数は命令を実行する上での処理の数と同じで決められた処理段階を経るため、自由に段数を変えられるのかがよく分からなかった。

質問とか感想

- 公式がいっぱい出てきましたが、テストでは実行サイクル数 C_r を命令数 N_i で正規化し、その逆数をとった公式を覚えていればなんとかなりますか？そこまでの導出も覚えておかないと厳しいでしょうか。

質問とか感想

- in-order実行では前の命令がストールすると後続命令も待たされることは理解できましたが、後続命令が「F D」まで進んでからストールするのか、「F」の直後でストールするのかの判断基準がまだ分かりません。

質問とか感想

- パイプライン段数を増やすとクロック周波数は上がりますが、ハザードや分岐予測ミスによってIPCが下がるため、必ずしも性能が向上するわけではないという理解で合っていますか？

質問とか感想

- 実際のCPUの改良(そんなに簡単なことではないと思いますが、、、)では、性能そのものと性能エネルギーのどちらをより重視するのでしょうか。

質問とか感想

- あと、段っていうと縦にみてしまう....横が段なの難しいですね

質問とか感想

- 命令とサイクル数のグラフが右下がりのグラフであることに何か意味はあるのでしょうか？

質問とか感想

- ○wayスーパスカラと○段の違いは何ですか。

質問とか感想

- テストは持ち込み有りですか。有りですね。ありがとうございます。

質問とか感想

- 数学が好きでハードウェアにも興味があるのですが、ハードウェアの研究をするうえで数学はどのくらい必要なものですか。

質問とか感想

- スライドを読んだだけでは計算があまり理解できなかったが、課題に取り組んで試行錯誤していくうちに少し理解を深めることができた。実践大事！

質問とか感想

- 実行サイクル数の式が、物理基礎で習った $x = x_0 + vt$ に似ているなと思いました。またパイプラインプロセッサでどうしても縦の列のみに着目してしまうので、横にも着目して、どれが今どの動きをしているのかをしっかりと確認しながら考えたいと思いました。

質問とか感想

- 課題7(7)をもう一度解説してほしい

質問とか感想

- 講義中に休憩の時間がある回とない回の違いは、講義内容のボリュームによるものですか。

質問とか感想

- 練習問題を用意していただいていることに今日気づきました。これらの問題の類題の他に単語の説明問題のようなものも期末試験では出題されますか。

質問とか感想

- 体感3割くらいしか理解していないような気がして、期末が不安でなりません…。重点的に勉強すべき点などなにかありますか？

- 第八回の課題についてなのですが、性能を測るためにはIPCを出した方がいいとスライドには書いてあったのですが、IPCを出す式には「全実行命令における何らかのハザードの出現率」を入れることができず、分岐予測ミスの出現率しか入れることができなかったもので頭がこんがらがりました。

質問とか感想

- 今回の講義では、「性能」について主に学習したが、普段日常で「この冷蔵庫の性能良いね」などを使う「性能」という単語とは意味が多少変わってくるような感じがし、困惑した。
- （日常における「性能」は、総合的な機能の良さを表し、コンピュータアーキテクチャの世界における「性能」は数学的に計算できる指標（実行時間の逆数）ということであっているでしょうか？）